XCPC 算法模板

1. 比赛注意 ---------- 4
2. 基础算法 ---------- 5
  2.1. 贪心 ---------- 5
    2.1.1. 反悔堆 ---------- 5
  2.2. 排序 ---------- 5
    2.2.1. 归并排序 ---------- 5
    2.2.2. 快速排序 ---------- 5
  2.3. 倍增 ---------- 6
    2.3.1. 应用 ---------- 6
    2.3.2. 例题 ---------- 6
  2.4. 二分/三分 ---------- 6
    2.4.1. 二分 ---------- 6
    2.4.2. 三分 ---------- 7
      2.4.2.1. 整数 ---------- 7
      2.4.2.2. 实数 ---------- 8
  2.5. 前缀和/差分 ---------- 8
    2.5.1. 普通前缀和 ---------- 8
    2.5.2. 普通差分 ---------- 9
    2.5.3. 树上前缀和 ---------- 9
    2.5.4. 树上差分 ---------- 10
    2.5.5. 应用 ---------- 11
  2.6. 扫描线 ---------- 13
  2.7. 高精度 ---------- 13
  2.8. 二进制 ---------- 24
    2.8.1. 二进制枚举子集 ---------- 24
3. 数据结构 ---------- 24
  3.1. 单调栈 ---------- 24
    3.1.1. 应用 ---------- 24
    3.1.2. 例题 ---------- 24
  3.2. 单调队列 ---------- 26
  3.3. 并查集 ---------- 26
    3.3.1. 带权并查集 ---------- 26
  3.4. 字典树 ---------- 27
    3.4.1. 模板 ---------- 27
  3.5. 树状数组 ---------- 29
    3.5.1. 优缺点 ---------- 29
    3.5.2. 单点修改 ---------- 29
    3.5.3. 建树 ---------- 29
    3.5.4. 求前缀和 ---------- 30

3.5.5. 应用 ---------- 30
3.5.6. 封装模板 ---------- 31
3.5.7. 高维树状数组 ---------- 32
3.5.8. PS ---------- 33
3.6. 线段树 ---------- 33
3.6.1. 经典模型 ---------- 33
3.6.2. 建树 ---------- 34
3.6.3. 单点修改 ---------- 34
3.6.4. 区间查询 ---------- 35
3.6.5. 区间修改 ---------- 35
3.6.6. 封装模板 ---------- 35
3.6.7. 动态开点线段树 ---------- 40
3.6.8. 势能线段树/吉司机线段树 ---------- 42
3.6.9. 线段树分裂 ---------- 42
3.6.10. 线段树合并 ---------- 42
3.6.11. 优化 ---------- 42
3.6.12. ps ---------- 43
3.6.13. 经典模型 ---------- 43
3.7. st 表 ---------- 45
3.8. 可持久化数据结构 ---------- 46
3.8.1. 可持久化线段树 ---------- 46
4. 数学 ---------- 54
4.1. 数论 ---------- 55
4.1.1. 数论基础 ---------- 55
4.1.2. 最大公约数 ---------- 55
4.1.2.1. 更相减损术 ---------- 55
4.1.2.2. 欧几里得算法 ---------- 56
4.1.2.3. 裴蜀定理 ---------- 56
4.1.2.4. 拓展欧几里得算法 ---------- 56
4.1.3. 乘法逆元 ---------- 56
4.1.4. 筛法 ---------- 57
4.1.4.1. 线性筛 ---------- 57
4.2. 组合数学 ---------- 57
4.2.1. 排列组合 ---------- 58
4.2.1.1. 求组合数 ---------- 58
4.2.1.2. 插板法 ---------- 59
4.2.1.3. 组合数性质/二项式推论 ---------- 59
4.2.2. 容斥原理 ---------- 60
4.2.3. 斯特林数 ---------- 60
4.2.4. 范德蒙卷积 ---------- 60

4.2.5. 卡特兰数 ---------- 60
4.3. 博弈论 ---------- 61
4.4. 线性代数 ---------- 61
4.4.1. 矩阵 ---------- 61
4.5. 高等数学 ---------- 62
4.5.1. 调和级数 ---------- 62
4.6. 结论 ---------- 62
5. 图论 ---------- 63
5.1. 基础概念 ---------- 63
5.1.1. 基环树 ---------- 63
5.2. 图的存储 ---------- 63
5.3. 图论 DFS ---------- 63
5.4. 树上问题 ---------- 64
5.4.1. 树的直径 ---------- 64
5.4.2. 最近公共祖先(Lowest Common Ancestor) ---------- 65
5.4.3. 虚树 ---------- 69
5.5. 最短路 ---------- 69
5.5.1. dijkstra ---------- 69
5.5.2. floyd ---------- 71
5.6. 最小生成树(Minimum Spanning Tree,MST) ---------- 71
5.7. 最短路与最小生成树的区别 ---------- 73
5.7.1. 最短路 ---------- 73
5.8. 拓扑排序(Topological sorting) ---------- 74
5.9. 二分图 ---------- 74
5.9.1. 二分图最大匹配 ---------- 74
5.10. 点双/边双 ---------- 75
5.10.1. 无向图 ---------- 75
5.10.2. tarjan ---------- 75
5.10.3. 有向图 ---------- 76
5.11. 点双/边双 ---------- 76
5.12. 强联通分量 ---------- 76
5.13. 网络流 ---------- 77
5.13.1. 定义 ---------- 77
5.14. 同余最短路 ---------- 77
5.15. 优化建图技巧 ---------- 78
5.15.1. 线段树优化建图 ---------- 78
6. 动态规划 ---------- 80
6.1. 背包 dp ---------- 80
6.1.1. 多重背包 ---------- 81
6.2. dp 的优化 ---------- 81

6.2.1. 斜率优化/凸壳优化 ---------- 81
7. 计算几何 ---------- 81
7.1. 基础知识 ---------- 81
7.1.1. 点 ---------- 81
7.1.2. 向量 ---------- 82
7.1.2.1. 点积(Dot product) ---------- 82
7.1.3. 直线方程 ---------- 82
7.1.4. 点积 ---------- 82
7.1.5. 叉积 ---------- 82
7.1.6. 极角 ---------- 83
8. 字符串 ---------- 84
8.1. 基础定义 ---------- 84
8.2. 哈希 ---------- 85
8.2.1. 单哈希 ---------- 85
8.2.2. 双哈希 ---------- 85
8.2.3. 带修哈希 ---------- 86
8.2.4. 树哈希 ---------- 86
8.3. KMP ---------- 86
8.3.1. kmp 求 border ---------- 86
8.3.2. kmp 匹配 ---------- 87
8.4. Manacher ---------- 87
9. 杂项 ---------- 88
9.1. 时间复杂度分析 ---------- 88
9.2. 随机 ---------- 88
9.3. 龟速乘 ---------- 88
9.4. 常数优化 ---------- 89
9.4.1. 快速取模 ---------- 89
9.4.2. 快读 ---------- 90
9.5. 离线算法 ---------- 90
9.5.1. 莫队 ---------- 90
9.5.1.1. 适用范围 ---------- 90
9.5.1.2. 时间复杂度 ---------- 90
9.5.1.3. 奇偶排序优化 ---------- 90
9.5.2. 整体二分 ---------- 92
9.5.3. 离线区间询问 ---------- 95
9.6. 置换环 ---------- 96
9.7. 开根号 ---------- 97
9.8. 矩阵旋转 ---------- 97

1. 比赛注意

2. 基础算法

2.1. 贪心

2.1.1. 反悔堆

2.2. 排序

2.2.1. 归并排序

```
void mergesort(int q[], int l, int r)
{
    if (l >= r)
        return;

    int mid = l + r >> 1;
    mergesort(q, l, mid);
    mergesort(q, mid + 1, r);
    int i = l, j = mid + 1, k = 0;
    while (i <= mid && j <= r)
    {
        if (q[i] <= q[j])
            tmp[k++] = q[i++];
        else
            tmp[k++] = q[j++];
    }
    while (i <= mid)
        tmp[k++] = q[i++];
    while (j <= r)
        tmp[k++] = q[j++];

    for (int i = l, j = 0; i <= r; i++, j++)
        q[i] = tmp[j];
}
```

2.2.2. 快速排序

```

```

基于快排的线性第 K 大

```
void solve()
{
    int n, k;
    cin >> n >> k;
    k++;
```

```
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    // 注意最小是第 0 小
    auto kth = [&](auto &kth, int l, int r) -> int
    {
        if (l == r && l == k)
            return a[k];
        int i = l, j = r;
        int tmp = a[l];
        while (i < j)
        {
            while (i < j && a[j] >= tmp)
                --j;
            if (i < j)
                swap(a[i], a[j]);
            while (i < j && a[i] <= tmp)
                ++i;
            if (i < j)
                swap(a[i], a[j]);
        }
        a[i] = tmp;
        if (i > k)
            return kth(kth, l, i - 1);
        else if (i < k)
            return kth(kth, i + 1, r);
        return a[k];
    };
    cout << kth(kth, 1, n) << endl;
}
```

2.3. 倍增

2.3.1. 应用

- 求 LCA
- 求 k 级祖先
- 维护路径信息(树上路径最小边权)

2.3.2. 例题

2.4. 二分/三分

2.4.1. 二分

```
int l = 1, r = n;
while (l < r)
{
    int mid = (l + r + 1) >> 1; // l+r>>1;
    if (check(mid))
        l = mid; // r=mid;
    else
        r = mid - 1; // l=mid+1;
}
```

2.4.2. 三分

2.4.2.1. 整数

整数的三分可能具有不确定性,可以通过改变 while 循环的条件为:

```
while(l+5<r)
```

来缩小范围,再通过

```
for(int i = l; i <= r; i++)
  ans = min(ans, calc(i));
for(int i = 1; i <= r; i++)
  ans = max(ans, calc(i));
//更新方式改成
l = lmid
r = rmid
```

来确定答案

凸函数的极大值

```
ll l, r;
while (l < r)
{
  ll lmid = l + (r - l) / 3;
  ll rmid = r - (r - l) / 3;
  if (calc(lmid) <= calc(rmid))
    l = lmid + 1;
  else
    r = rmid - 1;
}
printf("%lld\n", max(calc(l), calc(r)));
```

凹函数的极小值

```

```

```
  ll l, r;
  while (l < r)
  {
    ll lmid = l + (r - l) / 3;
    ll rmid = r - (r - l) / 3;
    if (calc(rmid) >= calc(lmid))
      r = mid - 1;
    else
      l = mid + 1;
  }
  printf("%lld\n", min(calc(l), calc(r)));
```

2.4.2.2. 实数

凸函数的极大值

```
db l, r;
for (int i = 0; i < 300; i++)
{
    db lmid = l + (r - l) / 3;
    db rmid = r - (r - l) / 3;
    if (calc(lmid) <= calc(rmid))
        l = lmid;
    else
        r = rmid;
}
printf("%.6f\n", calc(l))
```

凹函数的极小值

```
db l, r;
for (int i = 0; i < 300; i++)
{
    db lmid = l + (r - l) / 3;
    db rmid = r - (r - l) / 3;
    if (calc(rmid) >= calc(lmid))
        r = rmid;
    else
        l = lmid;
}
printf("%.6f\n", calc(l));
```

- PS 三分要求在极小值左边和右边都是严格单调,三分整数时要注意

2.5. 前缀和/差分

2.5.1. 普通前缀和

- 一维

```
s[i] = s[i - 1] + a[i];
```

- 二维

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        b[i][j] += b[i - 1][j] + b[i][j - 1] - b[i - 1][j - 1];
```

2.5.2. 普通差分

- 一维

```
void add(int l,int r, int w)//区间加
{
    s[l]+=w,s[r+1]-=w;
}
```

- 二维

```
void add(int x1, int y1, int x2, int y2, int w)//给左上角为 x1,y1, 右下角为 x2,y2 的矩形区域加 w
{
    b[x1][y1] += w;
    b[x2 + 1][y1] -= w;
    b[x1][y2 + 1] -= w;
    b[x2 + 1][y2 + 1] += w;
}
```

- 查询某一区域

```
int query(int x1, int y1, int x2, int y2) // 给左上角为 x1,y1, 右下角为 x2,y2 的矩形区域
{
    return s[x2][y2] - s[x1 - 1][y2] - s[x2][y1 - 1] + s[x1 - 1][y1 - 1];
}
```

2.5.3. 树上前缀和

- 点权

x,y 路径上的点权之和 s[x]+s[y]-s[lca]-s[f(lca)]

```

```

```
int query(int a, int b)
{
    int u=lca(a,b);
    return s[a] + s[b] - s[u] - s[f[u][0];
}
```

- 边权

把边权当作点权处理，边权加到深度更深的点上

```
(dist[u] > dist[v]) ? (s[u] += w, e[u] = w) : (s[v] += w, e[v] =
w);//一定要加括号！
```

x,y 路径上的边权和为 s[x]+s[y]−2*s[lca]

```
int query(int a, int b)
{
    return s[a] + s[b] - 2 * s[lca(a, b)];
}
```

2.5.4. 树上差分

- 点权

x,y 路径上所有点+w

d[x]+=w,d[y]+=w,d[lca]-=w,d[fa(lca)]-=w

```
void modify(int a, int b,int w)
{
    int u=lca(a,b);
    d[a]+=w, d[b]+=w, d[u] -= w,d[f[u][0] -= w;
}
```

- 边权

x,y 路径上所有边+w

d[x]+=w,d[y]+=w,d[lca]-=2*w

```
void modify(int a, int b, int w)
{
    d[a] += w, d[b] += w, d[lca(a, b)] -= 2 * w;
}
```

差分后做前缀和求答案

```
void cal(int u, int fa)
{
    for (auto to : edge[u])
        if (to != fa)
        {
            cal(to, u);
            d[u] += d[to];
        }
}
```

- 利用 dfs 序进行树上差分

2.5.5. 应用

- 利用前缀和降低维度

Looking for Waldo

在给定字母矩阵中找到同时含有 WALDO 的最小矩阵，输出面积

```
int n, m;
int cnt[10];
int inv[N];
map<char, int> mp;
void solve()
{
    mp['W'] = 1;
    mp['A'] = 2;
    mp['L'] = 3;
    mp['D'] = 4;
    mp['O'] = 5;
    cin >> n >> m;
    ll ans = LLONG_MAX;
    vector<vector<char>> a(max(n, m) + 10, vector<char>(min(n, m) +
10, '0'));
    if (n < m)
    {
        for (int i = n; i >= 1; i--)
            for (int j = 1; j <= m; j++)
                cin >> a[j][i];
        swap(n, m);
    }
    else
    {
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= m; j++)
                cin >> a[i][j];
    }
```

```
    for (int l = 1; l <= m; l++)
    {
        for (int i = 1; i <= n; i++)
            inv[i] = 0;
        for (int r = l; r <= m; r++)
        {
            for (int j = 1; j <= n; j++)
            {
                if (a[j][r] == 'W' || a[j][r] == 'A' || a[j][r] ==
'L' || a[j][r] == 'D' || a[j][r] == 'O')
                    inv[j] |= (1 << mp[a[j][r]]);
            }
            // 双指针求满足条件的最小区间
            memset(cnt, 0, sizeof cnt);
            int le = 1, now = 0;
            for (int ri = 1; ri <= n; ri++) // 枚举右端点
            {
                for (int k = 1; k <= 5; k++)
                {
                    if ((inv[ri] >> k) & 1)
                    {
                        cnt[k]++;
                        if (cnt[k] == 1)
                            now++;
                    }
                }
                while (now == 5)
                {
                    ll res = (r - l + 1) * (ri - le + 1);
                    ans = min(ans, res);
                    for (int k = 1; k <= 5; k++)
                    {
                        if ((inv[le] >> k) & 1)
                        {
                            cnt[k]--;
                            if (cnt[k] == 0)
                                now--;
                        }
                    }
                    le++;
                }
            }
        }
    }
    if (ans > n * m)
        cout << "impossible" << endl;
```

```
    else
        cout << ans << endl;
}
```

2.6. 扫描线

2.7. 高精度

1.自用

```
struct bign {
    vector<short> a;
    int sign;
    bign() : sign(1) {}
    bign(ll num) : sign(1) { *this = num; }
    bign(string &s) : sign(1) {
        for (int i = (int)s.size() - 1; i >= 0; i--)
            a.push_back(s[i] - '0');
    }
    bign &up() {
        while (a.size() > 1 && !a.back())
            a.pop_back(); // 去除前导零
        for (int i = 1; i < a.size(); i++) {
            a[i] += a[i - 1] / 10;
            a[i - 1] %= 10;
        }
        while (a.back() >= 10) {
            a.push_back(a.back() / 10);
            a[(int)a.size() - 2] %= 10;
        }
        return *this;
    }
    bign operator=(const bign T) {
        a = T.a;
        sign = T.sign;
        return *this;
    }
    bign operator=(ll num) {
        sign = num < 0 ? -1 : 1;
        num = abs(num);
        a.clear();
        while (num)
            a.push_back(num % 10), num /= 10;
        return *this;
    }
    bign &operator+=(const bign &T) {
```

```
        if (a.size() < T.a.size())
            a.resize(T.a.size());
        for (int i = 0; i < T.a.size(); i++)
            a[i] += T.a[i];
        return this->up();
    }
    bign operator+(const bign &T) const {
        if (this->sign != T.sign) {
            if (this->sign == -1) {
                return T + *this;
            } else {
                return *this - T;
            }
        }
        bign ans;
        ans.sign = T.sign;
        ans.a.resize(max(a.size(), T.a.size()));
        for (int i = 0; i < max(a.size(), T.a.size()); i++) {
            if (i < a.size())
                ans.a[i] += a[i];
            if (i < T.a.size())
                ans.a[i] += T.a[i];
        }
        return ans.up();
    }
    bign operator*(const bign &T) const {
        bign ans;
        ans.a.resize(a.size() + T.a.size());
        for (int i = 0; i < T.a.size(); i++)
            for (int j = 0; j < a.size(); j++)
                ans.a[i + j] += (a[j] * T.a[i]);
        return ans.up();
    }
    bign operator-(const bign &T) const {
        bign ans;
        if (*this < T) {
            ans = T - *this;
            ans.sign = -1;
        } else {
            for (int i = 0, t = 0; i < a.size(); i++) {
                t = a[i] - t;
                if (i < T.a.size())
                    t -= T.a[i];
                ans.a.push_back((t + 10) % 10);
                if (t < 0)
                    t = 1;
```

```
                else
                    t = 0;
            }
        }
        return ans.up();
    }
    bign operator/(const bign &T) const {
        ll i, j;
        bign ans = 0, c = *this;
        for (i = (int)a.size() - 1; i >= 0; i--) {
            ans = ans * 10 + a[i];
            for (j = 0; j < 10; j++)
                if (ans < T * (j + 1))
                    break;
            c.a[i] = j;
            ans = ans - T * j;
        }
        return c.up();
    }
    bign operator%(const bign &T) {
        int i, j;
        bign ans = 0;
        for (i = (int)a.size() - 1; i >= 0; i--) {
            ans = ans * 10 + a[i];
            for (j = 0; j < 10; j++)
                if (ans < T * (j + 1))
                    break;
            ans = ans - T * j;
        }
        return ans;
    }
    bool operator<(const bign &T) const {
        if (a.size() != T.a.size())
            return a.size() < T.a.size();
        for (int i = a.size() - 1; i >= 0; --i)
            if (a[i] != T.a[i])
                return a[i] < T.a[i];
        return 0;
    }
    bool operator>(const bign &T) const { return T < *this; }
    bool operator<=(const bign &b) const { return !(b < *this); }
    bool operator>=(const bign &b) const { return !(*this < b); }
    bool operator!=(const bign &b) const { return b < *this || *this
< b; }
    bool operator==(const bign &b) const { return !(b < *this) && !
(b > *this); }
```

```
    bool operator>(bign &T) {
        return T < *this;
    }
    friend istream &operator>>(istream &is, bign &T) {
        string s;
        is >> s;
        T.a.clear();
        if (s[0] == '-')
            T.sign = -1;
        for (int i = (int)s.size() - 1; i >= (T.sign != 1); i--)
            T.a.push_back(s[i] - '0');
        return is;
    }
    friend ostream &operator<<(ostream &os, const bign &T) {
        if (T.sign == -1)
            os << '-';
        for (int i = (int)T.a.size() - 1; i >= 0; i--)
            os << T.a[i];
        return os;
    }
};
```

2.

```
const int LEN = 100009;
struct BIGNUM {

    static const int BIT = 9;
    static const int MOD = 1e9; // 1eBIT 14
    bool flag;                  // 0 for 0 or positive, 1 for negative
    ll s[LEN];
    BIGNUM() {
        flag = 0;
        memset(s, 0, sizeof(s));
        s[0] = 1;
    }
    void init() { // 数组清零
        memset(s, 0, sizeof(s));
    }
    void print() { // 打印
        BIGNUM a = *this;
        if (a.flag) {
            printf("-");
        }
        printf("%lld", a.s[a.s[0]]);
        for (int i = a.s[0] - 1; i > 0; i--) {
```

```
        printf("%09lld", a.s[i]);
    }
    printf("\n");
}
BIGNUM operator=(const char *num) {
    if (num[0] == '-') {
        flag = 1;
        num++;
    } else if (num[0] == '+') {
        flag = 0;
        num++;
    }
    int l = strlen(num);
    s[0] = 0;
    for (int i = l - 1; i > -9; i -= 9) {
        int temp = 0;
        for (int j = i - 8; j <= i; j++) {
            if (j < 0)
                continue;
            else {
                temp = (temp << 1) + (temp << 3);
                temp += num[j] ^ 48;
            }
        }
        s[++s[0]] = temp;
    }
    while (!s[s[0]] && s[0] > 1)
        s[0]--;
    return *this;
}
BIGNUM operator=(const int num) {
    char a[LEN];
    sprintf(a, "%d", num);
    *this = a;
    return *this;
}
BIGNUM(int num) { *this = num; }
BIGNUM(const char *num) { *this = num; }
bool operator<(const BIGNUM &a) {
    if (flag > a.flag)
        return 1;
    else if (flag < a.flag)
        return 0;
    else {
        if ((s[0] < a.s[0] && (!flag)) || (s[0] > a.s[0] && flag))
            return 1;
        else if (s[0] != a.s[0])
            return 0;
```

```
        else {
            for (int i = s[0]; i > 0; i--) {
                if (s[i] < a.s[i]) {
                    if (flag)
                        return 0;
                    else
                        return 1;
                } else if (s[i] > a.s[i]) {
                    if (flag)
                        return 1;
                    else
                        return 0;
                }
            }
            return 0;
        }
    }
}
bool operator>(const BIGNUM &a) {
    if (flag > a.flag)
        return 0;
    else if (flag < a.flag)
        return 1;
    else {
        if ((s[0] > a.s[0] && (!flag)) || (s[0] < a.s[0] && flag))
            return 1;
        else if (s[0] != a.s[0]) {
            return 0;
        } else {
            for (int i = s[0]; i > 0; i--) {
                if (s[i] > a.s[i]) {
                    if (flag)
                        return 0;
                    else
                        return 1;
                } else if (s[i] < a.s[i]) {
                    if (flag)
                        return 1;
                    else
                        return 0;
                }
            }
            return 0;
        }
    }
}
bool operator==(const BIGNUM &a) {
    if (flag != a.flag)
```

```
            return 0;
        else {
            if (s[0] != a.s[0])
                return 0;
            else {
                for (int i = s[0]; i > 0; i--) {
                    if (s[i] != a.s[i])
                        return 0;
                }
                return 1;
            }
        }
    }
    bool operator>=(const BIGNUM &a) {
        if (*this > a || *this == a)
            return 1;
        return 0;
    }
    bool operator<=(const BIGNUM &a) {
        return (*this == a) || (*this < a);
    }
    int cmp_abs(const BIGNUM &a) { // 1 for this > a, 0 for this == a,
-1 for this < a
        if (s[0] > a.s[0])
            return 1;
        else if (s[0] < a.s[0])
            return -1;
        else {
            for (int i = s[0]; i > 0; i--) {
                if (s[i] > a.s[i])
                    return 1;
                else if (s[i] < a.s[i])
                    return -1;
            }
        }
        return 0;
    }
    BIGNUM operator+(const BIGNUM &B) {
        BIGNUM c, a = *this, b = B;
        if (!(a.flag || b.flag) || (a.flag && b.flag)) {
            c.flag = a.flag;
            int x = 0;
            c.s[0] = max(a.s[0], b.s[0]) + 1;
            for (int i = 1; i <= c.s[0]; i++) {
                c.s[i] = a.s[i] + b.s[i] + x;
                x = c.s[i] / MOD;
                c.s[i] %= MOD;
            }
```

```
} else if (a.flag) {          // a < 0
    if (a.cmp_abs(b) == 1) { //|a| > |b|, |ans| = |a| - |b|
        c.flag = 1;
        c.s[0] = max(a.s[0], b.s[0]) + 1;
        for (int i = 1; i <= c.s[0]; i++) {
            c.s[i] = a.s[i] - b.s[i];
            if (c.s[i] < 0) {
                c.s[i] += MOD;
                a.s[i + 1]--;
            }
        }
    } else { //|a| <= |b|, |ans| = |b| - |a|
        c.flag = 0;
        c.s[0] = max(a.s[0], b.s[0]) + 1;
        for (int i = 1; i <= c.s[0]; i++) {
            c.s[i] = b.s[i] - a.s[i];
            if (c.s[i] < 0) {
                c.s[i] += MOD;
                b.s[i + 1]--;
            }
        }
    }
} else {                      // b < 0
    if (b.cmp_abs(a) == 1) { // |b| > |a|, |ans| = |b| - |a|
        c.flag = 1;
        c.s[0] = max(a.s[0], b.s[0]) + 1;
        for (int i = 1; i <= c.s[0]; i++) {
            c.s[i] = b.s[i] - a.s[i];
            if (c.s[i] < 0) {
                c.s[i] += MOD;
                b.s[i + 1]--;
            }
        }
    } else { //|b| <= |a|, |ans| = |a| - |b|
        c.flag = 0;
        c.s[0] = max(a.s[0], b.s[0]) + 1;
        for (int i = 1; i <= c.s[0]; i++) {
            c.s[i] = a.s[i] - b.s[i];
            if (c.s[i] < 0) {
                c.s[i] += MOD;
                a.s[i + 1]--;
            }
        }
    }
}
while (c.s[c.s[0]] == 0 && c.s[0] > 1)
    c.s[0]--;
return c;
```

```
}
BIGNUM operator+=(const BIGNUM &a) {
    *this = *this + a;
    return *this;
}
BIGNUM operator-(const BIGNUM &B) {
    BIGNUM c, a = *this, b = B;
    if ((!a.flag) && (!b.flag)) {
        if (a >= b) {
            c.flag = 0;
            c.s[0] = max(a.s[0], b.s[0]) + 1;
            for (int i = 1; i <= c.s[0]; i++) {
                c.s[i] = a.s[i] - b.s[i];
                if (c.s[i] < 0) {
                    c.s[i] += MOD;
                    a.s[i + 1]--;
                }
            }
        } else {
            c.flag = 1;
            c.s[0] = max(a.s[0], b.s[0]) + 1;
            for (int i = 1; i <= c.s[0]; i++) {
                c.s[i] = b.s[i] - a.s[i];
                if (c.s[i] < 0) {
                    c.s[i] += MOD;
                    b.s[i + 1]--;
                }
            }
        }
    } else {
        b.flag = !b.flag;
        c = a + b;
    }
    while (c.s[c.s[0]] == 0 && c.s[0] > 1)
        c.s[0]--;
    return c;
}
BIGNUM operator-=(const BIGNUM &a) {
    *this = *this - a;
    return *this;
}
BIGNUM operator*(const BIGNUM &B) {
    BIGNUM c, a = *this, b = B;
    c.init();
    if (!(a.flag || b.flag) || (a.flag && b.flag))
        c.flag = 0;
    else {
        if ((a.s[0] == 1 && a.s[1] == 0) || (b.s[0] == 1 && b.s[1]
```

```
== 0))
                c = 0;
            else
                c.flag = 1;
        }
        c.s[0] = a.s[0] + b.s[0];
        for (int i = 1; i <= a.s[0]; i++) {
            int x = 0;
            for (int j = 1; j <= b.s[0]; j++) {
                c.s[i + j - 1] += a.s[i] * b.s[j] + x;
                x = c.s[i + j - 1] / MOD;
                c.s[i + j - 1] %= MOD;
            }
            c.s[i + b.s[0]] = x;
        }
        while (c.s[c.s[0]] > 0)
            c.s[0]++;
        while (c.s[c.s[0]] == 0 && c.s[0] > 1)
            c.s[0]--;
        return c;
    }
    BIGNUM operator*=(const BIGNUM &a) {
        *this = *this * a;
        return *this;
    }
    BIGNUM operator<<(const int &a) {
        for (int i = 0; i < a; i++) {
            s[0]++;
            for (int j = s[0]; j > 0; j--) {
                s[j] <<= 1;
            }
            for (int j = 1; j < s[0]; j++) {
                if (s[j] >= MOD) {
                    s[j] -= MOD;
                    s[j + 1]++;
                }
            }
            while (s[s[0]] == 0 && s[0] > 1)
                s[0]--;
        }
        return *this;
    }
    BIGNUM operator<<=(const int &a) {
        *this = *this << a;
        return *this;
    }
    BIGNUM operator>>(const int &a) {
        for (int i = 0; i < a; i++) {
```

```
            for (int j = s[0]; j > 0; j--) {
                if ((s[j] & 1) && j != 1)
                    s[j - 1] += MOD;
                s[j] >>= 1;
            }
        }
        while (s[s[0]] == 0 && s[0] > 1)
            s[0]--;
        return *this;
    }
    BIGNUM operator>>=(const int &a) {
        *this = *this >> a;
        return *this;
    }
    BIGNUM operator/(const BIGNUM &c) {
        BIGNUM a = *this, b = c, temp, ans;
        ans.init();
        temp.init();
        ans = 0;
        temp = 1;
        bool sign;
        if (!(a.flag || b.flag) || (a.flag && b.flag) || cmp_abs(c) ==
-1)
            sign = 0;
        else
            sign = 1;
        a.flag = b.flag = 0;
        while (a >= b) {
            b <<= 1;
            temp <<= 1;
        }
        while (temp.s[0] > 1 || temp.s[1]) {
            if (a >= b) {
                a -= b;
                ans += temp;
            }
            b >>= 1;
            temp >>= 1;
        }
        while (!ans.s[ans.s[0]] && ans.s[0] > 1)
            ans.s[0]--;
        ans.flag = sign;
        return ans;
    }
    BIGNUM operator/=(const BIGNUM &a) {
        *this = *this / a;
        return *this;
    }
```

```
    BIGNUM operator%(const BIGNUM &a) {
        BIGNUM c = *this / a, d = c * a;
        return *this - d;
    }
    BIGNUM operator%=(const BIGNUM &a) {
        *this = *this % a;
        return *this;
    }
};
ostream &operator<<(ostream &out, const BIGNUM &a) {
    if (a.flag) {
        out << '-';
    }
    out << a.s[a.s[0]];
    for (int i = a.s[0] - 1; i >= 1; i--)
        out << fixed << setfill('0') << setw(9) << a.s[i];
    return out;
}
istream &operator>>(istream &in, BIGNUM &a) {
    char str[LEN];
    in >> str;
    a = str;
    return in;
}
```

2.8. 二进制

2.8.1. 二进制枚举子集

```
for (int i = n; i; i = (i - 1) & n)//枚举 n 的子集
```

3. 数据结构

3.1. 单调栈

3.1.1. 应用

- 解决 NGE(Next Greater Element)问题
- 两元素间所有元素均(不)大/小于这两者问题

3.1.2. 例题

[模板]单调栈

```
int n, m;
int a[N], ans[N];
void solve()
```

```
{
    cin >> n;
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    stack<int> s;
    for (int i = 1; i <= n; i++)
    {
        while (s.size() && a[s.top()] < a[i])
        {
            ans[s.top()] = i;
            s.pop();
        }
        s.push(i);
    }
    for (int i = 1; i <= n; i++)
        cout << ans[i] << " ";
}
```

Patrik 音乐会的等待

```
int n, m;
int h[N];
void solve()
{
    cin >> n;
    for (int i = 1; i <= n; i++)
        cin >> h[i];
    stack<pii> s;// 这里 pair 的第二个成员表示相同元素的数量
    ll ans = 0;
    for (int i = 1; i <= n; i++)
    {
        int num = 0;
        while (s.size() && s.top().x <= h[i])
        {
            if (s.top().x == h[i])
                num = s.top().y;
            ans += s.top().y;
            s.pop();
        }
        if (s.size())
            ans++;
        s.push({h[i], num + 1});
    }
    cout << ans << endl;
}
```

3.2. 单调队列

滑动窗口 / 【模板】单调队列

给出一个长度为 n 的数组，输出每 k 个连续的数中的最大值和最小值。

```
int n, k;
int a[N], q[N];
void solve()
{
    cin >> n >> k;
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    int l = 1, r = 0;
    for (int i = 1; i <= n; i++)
    {
        while (l <= r && a[i] <= a[q[r]])
            r--;
        q[++r] = i;
        while (q[l] <= i - k)
            l++;
        if (i >= k)
            cout << a[q[l]] << " ";
    }
    cout << endl;
    l = 1, r = 0;
    for (int i = 1; i <= n; i++)
    {
        while (l <= r && a[i] >= a[q[r]])
            r--;
        q[++r] = i;
        while (q[l] <= i - k)
            l++;
        if (i >= k)
            cout << a[q[l]] << " ";
    }
    cout << endl;
}
```

3.3. 并查集

3.3.1. 带权并查集

The Third Letter

已知若干对士兵的相对位置,询问是否合法

```

```

```
void solve()
{
    int n, m;
    cin >> n >> m;
    vector<ll> p(n + 1), d(n + 1);
    for (int i = 1; i <= n; i++)
        p[i] = i, d[i] = 0;
    auto find = [&](const auto &find, ll x) -> ll
    {
        if (x != p[x])
        {
            ll res = find(find, p[x]);
            d[x] += d[p[x]];
            p[x] = res;
        }
        return p[x];
    };
    bool ok = true;
    for (int i = 1; i <= m; i++)
    {
        ll u, v, w;
        cin >> u >> v >> w;
        ll fa = find(find, u), fb = find(find, v);
        if (fa != fb)
        {
            p[fa] = fb;
            d[fa] = d[v] - d[u] + w;
        }
        else if (ok && d[u] - w != d[v])
        {
            ok = false;
            cout << "NO" << endl;
        }
    }
    if (ok)
        cout << "YES" << endl;
}
```

3.4. 字典树

3.4.1. 模板

```
struct Trie
{
    int root, tot, nex[N * M][2];
    ll val[N * M]; // N 输入的字符串的长度和 ,M 字符集
```

```
    int newnode()
    {
        memset(nex[tot], 0, sizeof nex[tot]);
        val[tot] = 0;
        return tot++;
    }
    void init()
    {
        memset(nex[0], 0, sizeof nex[0]);
        val[0] = 0;
        tot = 1;
        root = newnode();
    }
    void upd(ll x, ll v)
    {
        int id = root;
        for (int i = 30; ~i; i--)
        {

            int t = (x >> i) & 1;
            if (!nex[id][t])
                nex[id][t] = newnode();
            id = nex[id][t];
            val[id] = (val[id] + v) % mod;
        }
    }
    ll ask(ll x, ll k)
    {
        int id = root;
        ll res = 0;
        for (int i = 30; ~i; i--)
        {
            int t = (x >> i) & 1;
            if (((k >> i) & 1) == 1)
            {
                if (nex[id][t])
                    res = (res + val[nex[id][t]]) % mod;
                id = nex[id][t ^ 1];
            }
            else
                id = nex[id][t];
        }
        res = (res + val[id]) % mod;
        return res;
    }
} tr;
```

3.5. 树状数组

3.5.1. 优缺点

功能是线段树的子集，但是代码与常数相对线段树都较小,优点是代码量少.

3.5.2. 单点修改

时间复杂度:$O(\log N)$

```
void modify(int x, int s)//a[i]+=s
{
    for (; x <= n; x += x & (-x))
        c[x] += s;
}
```

3.5.3. 建树

- 可以通过单点修改来建树,但是这样是$O(\log n)$的.
- 每个 c[i]分管的范围是[x-lowbit(x),x]

对于每个右端点 R,其树状数组的长度是 lowbit(R),所以可以通过前缀和实现$O(n)$建树

```
void build()
{
    for (int i = 1; i <= n; ++i)
    {
        // c[i] = [x-lowbit(x)+1,x]
        // s 是前缀和数组
        c[i] = s[i] - s[i - lowbit(i)];
    }
}
```

- 暴力建树之所以会超时,是因为多次修改了重复节点,但是注意到每个 c[i]的值都是由更小的 c[i]得到的,所以可以通过从小到大的顺序,由子节点更新父节点,这样每个节点只会更新一次,这样也是 O(n)的了.

```
void build()
{
    for (int i = 1; i <= n; ++i)
    {
        c[i] += a[i];
        int fa = i + lowbit(i);
        if (fa <= n)
            c[fa] += c[i];
```

```
    }
}
```

3.5.4. 求前缀和

时间复杂度:$O(\log N)$

```
ll query(int x)
{
    ll res = 0;
    for (; x; x -= x & (-x))
        res += c[x];
    return res;
}
```

3.5.5. 应用

- 单点修改，区间查询

因为树状数组的查询是前缀和，所以只需要输出 query (r) - query (l-1)

```
bit.add(pos, x);
cout << bit.query(r) - bit.query(l - 1) << endl;
```

- 单点查询，区间修改

树状数组维护差分数组，每次区间修改视为普通的差分修改，单点查询时直接输出 query(pos)

```
//区间修改
bit.sum += x;//sum 相当于 a[1]
bit.add(i, x);
bit.add(i + 1, -x);
//单点查询
cout << bit.query(pos) << endl;
```

- 区间修改，区间查询

用两个树状数组分别维护数组的差分$c[i]$以及$c[i]*i$

```
void add(int l, int r, ll d)
{
    bit1.add(l, d);
    bit1.add(r + 1, -d);
    bit2.add(l, l * d);
    bit2.add(r + 1, (r + 1) * -d);
}
```

```
ll query(int l, int r)
{
    return (r + 1) * bit1.query(r) - bit2.query(r) - (l*bit1.query(l
- 1) - bit2.query(l - 1));
}
```

- 树状数组二分

```
int query(ll s)
{
    int pos = 0;
    for (int j = 18; j >= 0; j--)
        if (pos + (1 << j) <= n && c[pos + (1 << j)] <= s)
        {
            pos += (1 << j);
            s -= c[pos];
        }
    return pos;
}
```

- 求逆序对

```
for (int i = 1; i <= n; i++)
{
    cin >> a[i];
    a[i] = n + 1 - a[i];
    ans += query(a[i]);
    modify(a[i], 1);
}
```

3.5.6. 封装模板

```
struct BIT
{
    int n;
    vector<ll> a;
    BIT(int _n) : n(_n), a(n + 1) {}
    int lb(int x) { return x & -x; }
    void build(int n, vector<int> &s)
    {
        for (int i = 1; i <= n; i++)
        {
            a[i] += s[i];
            int fa = i + lb(i);
            if (fa <= n)
```

```
                a[fa] += a[i];
        }
    }
    void add(int x, ll y)
    {
        for (; x <= n; x += lb(x))
            a[x] += y;
    }
    ll query(int x)
    {
        ll res = 0;
        for (; x; x ^= lb(x))
            res += a[x];
        return res;
    }
    int search(ll s)//第一个小于等于 s 的位置
    {
        int pos = 0;
        for (int j = 20; j >= 0; j--)
            if (pos + (1 << j) <= n && a[pos + (1 << j)] <= s)
            {
                pos += (1 << j);
                s -= a[pos];
            }
        return pos;
    }
};
```

3.5.7. 高维树状数组

```
void modify(int x, int y, ll s)
{
    for (int p = x; p <= n; p += p & (-p))
        for (int q = y; q <= m; q += q & (-q))
            c[p][q] += s;
}
ll query(int x, int y)
{
    ll s = 0;
    for (int p = x; p; p -= p & (-p))
        for (int q = y; q; q -= q & (-q))
            s += c[p][q];
    return s;
}
```

3.5.8. PS

树状数组维护信息的下标只能从 1 开始!否则会造成死循环。

3.6. 线段树

3.6.1. 经典模型

1.二维数点

加入点要放在询问前面

坐标要加一防止出现 0

https://www.luogu.com.cn/problem/P2163

```
struct BIT
{
    int n;
    vector<int> a;
    BIT(int n) : n(n), a(n + 10) {}
    int lb(int x) { return x & -x; }
    void add(int x, ll s)
    {
        for (; x <= n; x += lb(x))
            a[x] += s;
    }
    ll query(int x)
    {
        ll res = 0;
        for (; x; x ^= lb(x))
            res += a[x];
        return res;
    }
};
void solve()
{
    int n, m;
    cin >> n >> m;
    BIT bit(1e7 + 10);
    vector<pii> p(n);
    vector<int> ans(m + 1, 0);
    vector<array<int, 5>> opt;
    for (int i = 1; i <= n; i++)
    {
        int x, y;
        cin >> x >> y;
        x++, y++;
        opt.push_back({x, y, 0, 0, 0});
```

```
    }
    for (int i = 1; i <= m; i++)
    {
        int x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        x1++, y1++, x2++, y2++;
        opt.push_back({x1 - 1, y1 - 1, 1, i, 1}); // x,y,操作类型, 答
案对应位置,+/-
        opt.push_back({x2, y1 - 1, 1, i, -1});
        opt.push_back({x1 - 1, y2, 1, i, -1});
        opt.push_back({x2, y2, 1, i, 1});
    }
    sort(opt.begin(), opt.end());
    for (auto &[x, y, v, p, t] : opt)
    {
        if (!v) // 加点
        {
            bit.add(y, 1);
        }
        else
        {
            ll res = bit.query(y);
            ans[p] += t * res;
        }
    }
    for (int i = 1; i <= m; i++)
        cout << ans[i] << endl;
}
```

3.6.2. 建树

```
void build(int id, int l, int r)
{
    if (l == r)
        seg[id].val = 0;
    else
    {
        int mid = l + r >> 1;
        build(id * 2, l, mid);
        build(id * 2 + 1, mid + 1, r);
        update(id);
    }
}
```

3.6.3. 单点修改

```
// 节点为 id,对应区间为[l,r],a[pos]->val
void change(int id, int l, int r, int pos, int val)
{
    if (l == r) // 叶子节点
        seg[id].val = val;
    else
    {
        int mid = l + r >> 1;
        if (pos <= mid)
            change(id * 2, l, mid, pos, val);
        else
            change(id * 2 + 1, mid + 1, r, pos, val);
        update(id);
    }
}
```

3.6.4. 区间查询

```
ll query(int id, int l, int r, int ql, int qr)
{
    if (l == ql && r == qr)
        return seg[id].val;
    int mid = l + r >> 1;
    if (qr <= mid)
        return query(id * 2, l, mid, ql, qr);
    else if (ql > mid)
        return query(id * 2 + 1, mid + 1, r, ql, qr);
    else
        return query(id * 2, l, mid, ql, mid) + query(id * 2 + 1,
mid + 1, r, mid + 1, qr);
}
```

3.6.5. 区间修改

在修改时设置懒标记，查询时将标记下放.

```

```

3.6.6. 封装模板

单点修改,区间查询

```
struct node
{
    ll pos, col;
};
```

```
struct segtree
{
    int n;
    vector<node> a;
    segtree(int _n) : n(_n * 4 + 10), a(n + 1) {}
    void update(int id)
    {
        a[id].pos = min(a[id * 2].pos, a[id * 2 + 1].pos);
        if (a[id * 2].pos < a[id * 2 + 1].pos)
            a[id].col = a[id * 2].col;
        else
            a[id].col = a[id * 2 + 1].col;
    }
    void build(int id, int l, int r, vector<int> &arr)
    {
        if (l == r)
            a[id].pos = arr[l], a[id].col = l;
        else
        {
            int mid = l + r >> 1;
            build(id * 2, l, mid, arr);
            build(id * 2 + 1, mid + 1, r, arr);
            update(id);
        }
    }
    void change(int id, int l, int r, int pos, int t)
    {
        if (l == r) // 叶子节点
            a[id].pos = t;
        else
        {
            int mid = l + r >> 1;
            if (pos <= mid)
                change(id * 2, l, mid, pos, t);
            else
                change(id * 2 + 1, mid + 1, r, pos, t);
            update(id);
        }
    }
    pii query(int id, int l, int r, int ql, int qr)
    {
        if (l == ql && r == qr)
            return {a[id].pos, a[id].col};
        int mid = l + r >> 1;
        if (qr <= mid)
            return query(id * 2, l, mid, ql, qr);
```

```
        else if (ql > mid)
            return query(id * 2 + 1, mid + 1, r, ql, qr);
        else
        {
            auto t1 = query(id * 2, l, mid, ql, mid);
            auto t2 = query(id * 2 + 1, mid + 1, r, mid + 1, qr);
            if (t1.x < t2.x)
                return t1;
            return t2;
        }
    }
};
```

区间加,区间和

```
struct segtree
{
    struct node
    {
        ll t, val, sz;
    };
    int n;
    vector<node> a;
    segtree(int _n) : n(_n * 4 + 10), a(n + 1) {}
    void update(int id) { a[id].val = a[id * 2].val + a[id * 2 +
1].val; }
    void settag(int id, int t)
    {
        a[id].val = a[id].val + t * (a[id].sz);
        a[id].t = a[id].t + t;
    }
    void pushdown(int id)
    {
        if (a[id].t)
        {
            settag(id * 2, a[id].t);
            settag(id * 2 + 1, a[id].t);
            a[id].t = 0;
        }
    }
    void build(int id, int l, int r, vector<int> &arr)
    {
        a[id].t = 0;
        a[id].sz = r - l + 1;
        if (l == r)
            a[id].val = arr[l];
```

```
        else
        {
            int mid = l + r >> 1;
            build(id * 2, l, mid, arr);
            build(id * 2 + 1, mid + 1, r, arr);
            update(id);
        }
    }
    void modify(int id, int l, int r, int ql, int qr, int t)
    {
        if (r < ql || l > qr)
            return;
        if (l >= ql && r <= qr)
        {
            settag(id, t);
            return;
        }
        int mid = l + r >> 1;
        pushdown(id);
        if (ql <= mid)
            modify(id * 2, l, mid, ql, qr, t);
        if (qr > mid)
            modify(id * 2 + 1, mid + 1, r, ql, qr, t);
        update(id);
    }
    ll query(int id, int l, int r, int ql, int qr)
    {
        if (r < ql || l > qr)
            return 0ll;
        if (l >= ql && r <= qr)
            return a[id].val;
        int mid = l + r >> 1;
        pushdown(id);
        ll res = 0;
        if (ql <= mid)
            res += query(id * 2, l, mid, ql, qr);
        if (qr > mid)
            res += query(id * 2 + 1, mid + 1, r, ql, qr);
        return res;
    }
};
```

区间加,区间赋值,区间乘,区间查询

```
struct tag
{
```

```
    ll mul, add;
    tag operator+(const tag &t) const
    {
        return {mul * t.mul % mod, (add * t.mul + t.add) % mod};
    }
};
struct node
{
    tag t;
    ll val, sz;
};
struct segtree
{
    int n;
    vector<node> a;
    segtree(int _n) : n(_n * 4 + 10), a(n + 1) {}
    void update(int id) { a[id].val = (a[id * 2].val + a[id * 2 +
1].val) % mod; }
    void settag(int id, tag t)
    {
        a[id].val = (a[id].val * t.mul + a[id].sz * t.add) % mod;
        a[id].t = a[id].t + t;
    }
    void pushdown(int id)
    {
        if (a[id].t.mul != 1 || a[id].t.add) // 标记非空
        {
            settag(id * 2, a[id].t);
            settag(id * 2 + 1, a[id].t);
            a[id].t.add = 0, a[id].t.mul = 1;
        }
    }
    void build(int id, int l, int r, vector<int> &arr)
    {
        a[id].t = {1, 0};
        a[id].sz = r - l + 1;
        if (l == r)
            a[id].val = {arr[l]};
        else
        {
            int mid = l + r >> 1;
            build(id * 2, l, mid, arr);
            build(id * 2 + 1, mid + 1, r, arr);
            update(id);
        }
    }
```

```
    void modify(int id, int l, int r, int ql, int qr, tag t)
    {
        if (l == ql && r == qr)
        {
            settag(id, t);
            return;
        }
        int mid = l + r >> 1;
        pushdown(id);
        if (qr <= mid)
            modify(id * 2, l, mid, ql, qr, t);
        else if (ql > mid)
            modify(id * 2 + 1, mid + 1, r, ql, qr, t);
        else
        {
            modify(id * 2, l, mid, ql, mid, t);
            modify(id * 2 + 1, mid + 1, r, mid + 1, qr, t);
        }
        update(id);
    }
    ll query(int id, int l, int r, int ql, int qr)
    {
        if (l == ql && r == qr)
            return a[id].val;
        int mid = l + r >> 1;
        pushdown(id);
        if (qr <= mid)
            return query(id * 2, l, mid, ql, qr);
        else if (ql > mid)
            return query(id * 2 + 1, mid + 1, r, ql, qr);
        else
        {
            return (query(id * 2, l, mid, ql, mid) + query(id * 2 +
1, mid + 1, r, mid + 1, qr)) % mod;
        }
    }
};
```

3.6.7. 动态开点线段树

结点只有在有需要的时候才被创建

```
#define ls(x) (a[x].l)
#define rs(x) (a[x].r)
#define sum(x) a[x].sum
```

```
struct node
{
    ll sum, l, r;
};
struct segtree
{
    int n;
    vector<node> a;
    segtree(int _n) : n(0) { a.reserve(_n); }
    void add(ll &id, int l, int r, int pos, int t) // add
    {
        if (!id)
            id = ++n;
        sum(id) += t;
        if (l == r)
            return;
        int mid = l + r >> 1;
        if (pos <= mid)
            add(ls(id), l, mid, pos, t);
        else
            add(rs(id), mid + 1, r, pos, t);
    }
    void change(ll &id, int l, int r, int pos, int t) // change
    {
        if (!id)
            id = ++n;
        if (l == r)
        {
            sum(id) = t;
            return;
        }
        int mid = l + r >> 1;
        if (pos <= mid)
            change(ls(id), l, mid, pos, t);
        else
            change(rs(id), mid + 1, r, pos, t);
        sum(id) = sum(ls(id)) + sum(rs(id));
    }
    ll query(ll id, int l, int r, int ql, int qr)
    {
        if (!id || r < ql || l > qr)
            return 0;
        if (l >= ql && r <= qr)
            return sum(id);
        int mid = l + r >> 1;
        return query(ls(id), l, mid, ql, qr) + query(rs(id), mid +
```

```
1, r, ql, qr);
    }
};
```

3.6.8. 势能线段树/吉司机线段树

一般线段树的区间修改是基于懒标记的,但是有些问题,比如区间开根号,区间取模等,这些操作是基于叶子节点的值的,无法通过懒标记修改,只能暴力到叶子节点

这个时候就需要势能线段树,可以简单理解为,总操作数不多,所以暴力操作也不会超时

```
void modify(int id, int l, int r, int ql, int qr, ll t)
{
    if (l >= ql && r <= qr && a[id].mx < t)
        return;
    if (r < ql || l > qr)
        return;
    if (l == r)
    {
        ll res = a[id].val % t;
        a[id].val = a[id].mx = res;
        return;
    }
    int mid = l + r >> 1;
    if (ql <= mid)
        modify(id * 2, l, mid, ql, qr, t);
    if (qr > mid)
        modify(id * 2 + 1, mid + 1, r, ql, qr, t);
    update(id);
}
```

3.6.9. 线段树分裂

3.6.10. 线段树合并

===线段树优化建图

3.6.11. 优化

1. 在叶子节点处无需下放懒惰标记，所以懒标记可以不下传到叶子节点。

2.标记永久化：如果确定懒惰标记不会在中途被加到溢出（即超过了该类型数据所能表示的最大范围），那么就可以将标记永久化。标记永久化可以避免下传懒惰标记，只需在进行询问时把标记的影响加到答案当中，从而降低程序常数。具体如何处理与题目特性相关，需结合题目来写。这也是树套树和可持久化数据结构中会用到的一种技巧

3.6.12. ps

- 如果数组的大小在$2^{\{n\}}+1-2^{\{n+1\}}$之间，那么线段树的大小可能会达到$2^{\{n+2\}}$，最坏情况下是 4 倍的大小，所以要开 4 倍
- 用线段树解决问题需要满足能高效合并两个区间的信息
- 线段树维护信息的下标只能从 1 开始!
- 注意是否会出现$l>r$的情况,这种情况会无限递归
- 线段树需要支持能够高效合并两个区间
- 用 vector 实现线段树时，因为 vector 的存储是自动管理的，按需扩张收缩，vector 通常占用多于静态数组的空间，因为要分配更多内存以管理将来的增长。

可以使用 reserve 预先增大 vector 的容量。

3.6.13. 经典模型

带修区间最大子段和

https://www.luogu.com.cn/problem/SP1716

```
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef double db;
const ll mod = 998244353;
const int N = 5e5 + 10, M = 25;
random_device rd;        // 将用于为随机数引擎获得种子
mt19937_64 gen(time(0)); // 以播种标准 mersenne_twister_engine
struct node
{
    ll ls, rs, ms, s;
};
struct segtree
{
    int n;
    vector<node> a;
    segtree(int _n) : n(_n * 4 + 10), a(n + 1) {}
    void update(int id)
    {
        a[id].ls = max(a[id * 2].s + a[id * 2 + 1].ls, a[id *
2].ls);
        a[id].rs = max(a[id * 2 + 1].s + a[id * 2].rs, a[id * 2 +
```

```
1].rs);
        a[id].s = a[id * 2].s + a[id * 2 + 1].s;
        a[id].ms = max({a[id * 2].ms, a[id * 2 + 1].ms, a[id * 2].rs
+ a[id * 2 + 1].ls});
    }
    void build(int id, int l, int r, vector<int> &arr)
    {
        if (l == r)
            a[id].s = a[id].ms = a[id].ls = a[id].rs = arr[l];
        else
        {
            int mid = l + r >> 1;
            build(id * 2, l, mid, arr);
            build(id * 2 + 1, mid + 1, r, arr);
            update(id);
        }
    }
    void change(int id, int l, int r, int pos, int t)
    {
        if (l == r) // 叶子节点
            a[id].s = a[id].ms = a[id].ls = a[id].rs = t;
        else
        {
            int mid = l + r >> 1;
            if (pos <= mid)
                change(id * 2, l, mid, pos, t);
            else
                change(id * 2 + 1, mid + 1, r, pos, t);
            update(id);
        }
    }
    node query(int id, int l, int r, int ql, int qr)
    {
        if (l == ql && r == qr)
            return a[id];
        int mid = l + r >> 1;
        if (qr <= mid)
            return query(id * 2, l, mid, ql, qr);
        else if (ql > mid)
            return query(id * 2 + 1, mid + 1, r, ql, qr);
        else
        {
            auto t1 = query(id * 2, l, mid, ql, mid);
            auto t2 = query(id * 2 + 1, mid + 1, r, mid + 1, qr);
            node t;
            t.s = t1.s + t2.s;
```

```
            t.ls = max(t1.s + t2.ls, t1.ls);
            t.rs = max(t2.s + t1.rs, t2.rs);
            t.ms = max({t1.ms, t2.ms, t1.rs + t2.ls});
            return t;
        }
    }
};
void solve()
{
    int n, q;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    segtree seg(n);
    seg.build(1, 1, n, a);
    cin >> q;
    int opt, l, r;
    for (int i = 1; i <= q; i++)
    {
        cin >> opt >> l >> r;
        if (opt == 0)
            seg.change(1, 1, n, l, r);
        else
            cout << seg.query(1, 1, n, l, r).ms << endl;
    }
}
int main()
{
    // cout << setprecision(5);
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    // cin >> _;
    while (_--)
        solve();
    return 0;
}
```

3.7. st 表

```
struct ST
{
    int n, q;
    vector<int> lg;
    vector<vector<ll>> f;
```

```
    ST(int _n) : n(_n), lg(n + 3, 0), f(25, vector<ll>(n + 1, 0)) {}
    void init(vector<int> &a)
    {
        lg[1] = 0;
        for (int i = 2; i <= n; i++)
            lg[i] = lg[i / 2] + 1;
        for (int i = 1; i <= n; i++)
            f[0][i] = a[i];
        for (int j = 1; j <= 20; j++)
            for (int i = 1; i + (1 << j) - 1 <= n; i++)
                f[j][i] = max(f[j - 1][i], f[j - 1][i + (1 << (j -
1))]);
    }
    int query(int l, int r)
    {
        int len = lg[r - l + 1];
        // 或者写成__lg(r - l + 1);
        return max(f[len][l], f[len][r - (1 << len) + 1]);
    }
};
```

3.8. 可持久化数据结构

3.8.1. 可持久化线段树

每次单点操作，节点数量最多增加$\log(n)$

https://www.luogu.com.cn/problem/P3919

```
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
#define ls(x) (a[x].l)
#define rs(x) (a[x].r)
#define sum(x) a[x].sum
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef double db;
const ll mod = 1e9 + 7;
const int N = 7e7 + 10, M = 25;
random_device rd;
mt19937_64 gen(rd());
struct node
```

```
{
    ll sum, l, r;
};
struct segtree
{
    int n;
    vector<node> a;
    segtree(int _n) : n(1) { a.reserve(_n); }
    // segtree(int _n) : n(1), a(n) {}//会re
    void update(int id)
    {
        sum(id) = sum(ls(id)) + sum(rs(id));
    }
    void build(int id, int l, int r, vector<int> &arr)
    {
        if (l == r)
            sum(id) = arr[l];
        else
        {
            ls(id) = ++n, rs(id) = ++n;
            int mid = (l + r) / 2;
            build(ls(id), l, mid, arr);
            build(rs(id), mid + 1, r, arr);
            update(id);
        }
    }
    void change(int pos, int l, int r, int id, int nxt, int val)
    {
        if (l == r)
        {
            sum(nxt) = val;
            return;
        }
        ls(nxt) = ls(id), rs(nxt) = rs(id);
        int mid = (l + r) / 2;
        if (pos <= mid)
            ls(nxt) = ++n, change(pos, l, mid, ls(id), ls(nxt),
val);
        else
            rs(nxt) = ++n, change(pos, mid + 1, r, rs(id), rs(nxt),
val);
        update(nxt);
    }
    ll query(int id, int l, int r, int ql, int qr) // 区间查询
    {
        if (l == ql && r == qr)
```

```
            return sum(id);
        int mid = (l + r) / 2;
        if (qr <= mid)
            return query(ls(id), l, mid, ql, qr);
        else if (ql > mid)
            return query(rs(id), mid + 1, r, ql, qr);
        else
            return query(ls(id), l, mid, ql, mid) + query(rs(id),
mid + 1, r, mid + 1, qr);
    }
};
void solve()
{
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1), root(m + 10);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    segtree seg(n * 4 + m * (__lg(n) + 3));
    seg.build(1, 1, n, a);
    root[0] = 1;
    int nxt, pos, val, opt;
    for (int i = 1; i <= m; i++)
    {
        cin >> nxt >> opt >> pos;
        if (opt == 1)
        {
            cin >> val;
            root[i] = ++seg.n;
            seg.change(pos, 1, n, root[nxt], root[i], val);
        }
        else
        {
            root[i] = root[nxt];
            cout << seg.query(root[i], 1, n, pos, pos) << endl;
        }
    }
}
int main()
{
    // cout << setprecision(5);
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    // cin >> _;
    while (_--)
```

```
        solve();
    return 0;
}
```

如果需要区间修改，则可以使用标记永久化来节省空间

```

```

主席树

静态查询区间第 k 小

```
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
#define ls(x) (a[x].l)
#define rs(x) (a[x].r)
#define sum(x) a[x].sum
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef double db;
const ll mod = 1e9 + 7;
const int N = 7e7 + 10, M = 25;
random_device rd;
mt19937_64 gen(rd());
struct node
{
    ll sum, l, r;
};
struct segtree
{
    int n;
    vector<node> a;
    segtree(int _n) : n(1) { a.reserve(_n); }
    // segtree(int _n) : n(1), a(n) {}//会re
    void update(int id)
    {
        sum(id) = sum(ls(id)) + sum(rs(id));
    }
    void build(int id, int l, int r)
    {
        if (l == r)
            sum(id) = 0;
```

```
        else
        {
            ls(id) = ++n, rs(id) = ++n;
            int mid = (l + r) / 2;
            build(ls(id), l, mid);
            build(rs(id), mid + 1, r);
            update(id);
        }
    }
    void change(int pos, int l, int r, int id, int nxt, int val)
    {
        if (l == r)
        {
            sum(nxt) = sum(id) + val;
            return;
        }
        ls(nxt) = ls(id), rs(nxt) = rs(id);
        int mid = (l + r) / 2;
        if (pos <= mid)
            ls(nxt) = ++n, change(pos, l, mid, ls(id), ls(nxt),
val);
        else
            rs(nxt) = ++n, change(pos, mid + 1, r, rs(id), rs(nxt),
val);
        update(nxt);
    }
    ll query(int id, int l, int r, int ql, int qr) //某个版本的区间查询
    {
        if (l == ql && r == qr)
            return sum(id);
        int mid = (l + r) / 2;
        if (qr <= mid)
            return query(ls(id), l, mid, ql, qr);
        else if (ql > mid)
            return query(rs(id), mid + 1, r, ql, qr);
        else
            return query(ls(id), l, mid, ql, mid) + query(rs(id),
mid + 1, r, mid + 1, qr);
    }
    ll kth(int id, int nxt, int l, int r, int k)
    {
        if (l == r)
            return l;
        int res = sum(ls(nxt)) - sum(ls(id));
        int mid = l + r >> 1;
        if (res >= k)
```

```
            return kth(ls(id), ls(nxt), l, mid, k);
        else
            return kth(rs(id), rs(nxt), mid + 1, r, k - res);
    }
};
void solve()
{
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1), root(n + 1), b;
    for (int i = 1; i <= n; i++)
        cin >> a[i], b.push_back(a[i]);
    sort(b.begin(), b.end());
    b.erase(unique(b.begin(), b.end()), b.end());
    segtree seg(1e7);
    root[0] = 1;
    seg.build(root[0], 1, n);
    for (int i = 1; i <= n; i++)
    {
        root[i] = ++seg.n;
        int pos = lower_bound(b.begin(), b.end(), a[i]) - b.begin()
+ 1;
        seg.change(pos, 1, n, root[i - 1], root[i], 1);
    }
    int l, r, k;
    for (int i = 1; i <= m; i++)
    {
        cin >> l >> r >> k;
        int ans = seg.kth(root[l - 1], root[r], 1, n, k);
        cout << b[ans - 1] << endl;
    }
}
int main()
{
    // cout << setprecision(5);
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    // cin >> _;
    while (_--)
        solve();
    return 0;
}
```

查询区间小于等于 k 的元素的和

https://atcoder.jp/contests/abc339/tasks/abc339_g

```cpp
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
#define ls(x) (a[x].l)
#define rs(x) (a[x].r)
#define sum(x) a[x].sum
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef double db;
const ll mod = 1e9 + 7;
const int N = 7e7 + 10, M = 25;
random_device rd;
mt19937_64 gen(rd());
struct node
{
    ll sum, l, r;
};
struct segtree
{
    int n;
    vector<node> a;
    segtree(int _n) : n(1) { a.reserve(_n); }
    void update(int id)
    {
        sum(id) = sum(ls(id)) + sum(rs(id));
    }
    void build(int id, int l, int r)
    {
        if (l == r)
            sum(id) = 0;
        else
        {
            ls(id) = ++n, rs(id) = ++n;
            int mid = l + r >> 1;
            build(ls(id), l, mid);
            build(rs(id), mid + 1, r);
            update(id);
        }
    }
    void change(int pos, int l, int r, int id, int nxt, int val)
    {
        if (l == r)
        {
```

```
            sum(nxt) = sum(id) + val;
            return;
        }
        ls(nxt) = ls(id), rs(nxt) = rs(id);
        int mid = l + r >> 1;
        if (pos <= mid)
            ls(nxt) = ++n, change(pos, l, mid, ls(id), ls(nxt),
val);
        else
            rs(nxt) = ++n, change(pos, mid + 1, r, rs(id), rs(nxt),
val);
        update(nxt);
    }
    ll query(int id, int l, int r, int ql, int qr) // 查询某个版本的区
间和
    {
        if (l == ql && r == qr)
            return sum(id);
        int mid = l + r >> 1;
        if (qr <= mid)
            return query(ls(id), l, mid, ql, qr);
        else if (ql > mid)
            return query(rs(id), mid + 1, r, ql, qr);
        else
            return query(ls(id), l, mid, ql, mid) + query(rs(id),
mid + 1, r, mid + 1, qr);
    }
    ll query(int id, int nxt, int l, int r, int ql, int qr) // 在 id-
nxt 版本之间查询值域在 ql-qr 的和
    {
        if (ql > qr)
            return 0;
        if (l == ql && r == qr)
            return sum(nxt) - sum(id);
        int mid = l + r >> 1;
        if (qr <= mid)
            return query(ls(id), ls(nxt), l, mid, ql, qr);
        else if (ql > mid)
            return query(rs(id), rs(nxt), mid + 1, r, ql, qr);
        else
            return query(ls(id), ls(nxt), l, mid, ql, mid) +
query(rs(id), rs(nxt), mid + 1, r, mid + 1, qr);
    }
};

void solve()
```

```
{
    ll n, q, mx = 0;
    cin >> n;
    vector<ll> a(n + 1), b;
    for (int i = 1; i <= n; i++)
        cin >> a[i], b.push_back(a[i]), mx = max(mx, a[i]);

    sort(b.begin(), b.end());
    b.erase(unique(b.begin(), b.end()), b.end());
    cin >> q;
    vector<ll> root(n + 1);
    ll pre = 0;
    root[0] = 1;
    segtree seg(2e7);
    seg.build(root[0], 1, n);
    for (int i = 1; i <= n; i++)
    {
        root[i] = ++seg.n;
        int pos = (lower_bound(b.begin(), b.end(), a[i]) -
b.begin()) + 1;
        seg.change(pos, 1, n, root[i - 1], root[i], a[i]);
    }
    for (int i = 1; i <= q; i++)
    {
        ll l, r, val;
        cin >> l >> r >> val;
        l ^= pre, r ^= pre, val ^= pre;
        int pos = (upper_bound(b.begin(), b.end(), val) -
b.begin());
        pre = seg.query(root[l - 1], root[r], 1, n, 1, pos);
        cout << pre << endl;
    }
}
int main()
{
    // cout << setprecision(5);
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    // cin >> _;
    while (_--)
        solve();
    return 0;
}
```

4. 数学

4.1. 数论

4.1.1. 数论基础

$n \leq$	10^1	10^2	10^3	10^4	10^5	10^6	10^7	10^8	10^9
$\max\{\omega(n)\}$	2	3	4	5	6	7	8	8	9
$\max\{d(n)\}$	4	12	32	64	128	240	448	768	1344
$n \leq$	10^{10}	10^{11}	10^{12}	10^{13}	10^{14}	10^{15}	10^{16}	10^{17}	10^{18}
$\max\{\omega(n)\}$	10	10	11	12	12	13	13	14	15
$\max\{d(n)\}$	2304	4032	6720	10752	17280	26880	41472	64512	103680

Figure 1: 因子种类数与因子数表

质数一定满足:6n+1,6n-1

- 下取整除法

```
ll floor(ll x, ll m)
{
    ll r = (x % m + m) % m;
    return (x - r) / m;
}
```

如果只是正数,是可以直接 a/b 的,但如果 a 是负数,就需要上面那个

上取整除法可以写成 floor(x - 1, m)+1

4.1.2. 最大公约数

结合律:

$\gcd(a, b, c) = \gcd(\gcd(a, b), c)$

4.1.2.1. 更相减损术

时间复杂度

$O(\max(a, b))$

```
ll gcd(ll a, ll b)
{
    if (a == b)
        return a;
```

```
    return (a > b ? gcd(a - b, b) : gcd(a, b - a));
}
```

4.1.2.2. 欧几里得算法

时间复杂度

$O(\log(\max(a,b)))$

复杂度证明:

```
ll gcd(ll a, ll b) { return b ? gcd(b, a % b) : a; }
```

4.1.2.3. 裴蜀定理

$\gcd(a,b)|ax+by\exists x,y \leftrightarrow ax+by=\gcd(a,b)$

4.1.2.4. 拓展欧几里得算法

参考资料:https://zhuanlan.zhihu.com/p/647843184

如果$b \neq 0$,那么求出的可行解一定满足$|x|<b,|y|<a$

设 exgcd 得到的一组特解为x_0,y_0,那么存在通解为:

$x'=x_0+\frac{b}{\gcd(a,b)}t$

$y'=y_0-\frac{a}{\gcd(a,b)}t$

$t=...,-2,-1,0,1,2,..$

```
ll exgcd(ll a, ll b, ll &x, ll &y)
{
    if (!b)
    {
        x = 1, y = 0;
        return a;
    }
    ll d = exgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return d;
}
```

4.1.3. 乘法逆元

- 快速幂求逆元

快速幂法使用了费马小定理，要求 mod 是一个素数

```

```

```
ll qmi(ll a, ll b = mod - 2)
{
    ll res = 1;
    a %= mod;
    while (b)
    {
        if (b & 1)
            res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
```

- exgcd 求逆元

要求$\gcd(a, \mathrm{mod}) = 1$

```

```

4.1.4. 筛法

4.1.4.1. 线性筛

时间复杂度:$O(n)$

```
vector<int> pr;
vector<bool> not_pr(N);
auto getpr = [&](int n)
{
    for (int i = 2; i <= n; ++i)
    {
        if (!not_pr[i])
            pr.push_back(i);
        for (int p : pr)
        {
            if (i * p > n)
                break;
            not_pr[i * p] = true;
            if (i % p == 0) // 说明 i*p 已经被一个更小的 i 判断过了
                break;
        }
    }
};
```

4.2. 组合数学

4.2.1. 排列组合

4.2.1.1. 求组合数

- 求 1~n 的任意组合数

时间复杂度:$O(n)$

```
struct PC
{
    int mx;
    vector<ll> fac, invfac;
    ll qmi(ll a, ll b)
    {
        ll res = 1;
        while (b)
        {
            if (b & 1)
                res = res * a % mod;
            b >>= 1;
            a = a * a % mod;
        }
        return res;
    }
    PC(int n) : mx(n), fac(n + 10), invfac(n + 10)
    {
        fac[0] = 1;
        for (int i = 1; i <= n; i++)
            fac[i] = fac[i - 1] * i % mod;
        invfac[n] = qmi(fac[n], mod - 2);
        for (int i = n - 1; i >= 0; i--)
            invfac[i] = invfac[i + 1] * (i + 1) % mod;
    }
    ll C(int n, int m)
    {
        if (n < m || m < 0 || n > mx)
            return 0ll;
        return fac[n] * invfac[n - m] % mod * invfac[m] % mod;
    }
    ll A(int n, int m)
    {
        if (n < m || m < 0 || n > mx)
            return 0ll;
        return fac[n] * invfac[n - m] % mod;
    }
};
```

- 直接求组合数

```
auto C = [&](ll a, ll b) -> ll
{
    if (a < b)
        return 0ll;
    __int128_t res = 1;
    for (ll i = a, j = 1; j <= b; i--, j++)
        res = res * i / j;
    return (ll)res;
};
```

- 跟据组合数定义

```

```

4.2.1.2. 插板法

- n 个 相同 元素分成非空的 k 组

n-1 个空隙插入 k-1 个板子

$$\binom{n-1}{k-1}$$

- n 个 相同 元素分成 k 组(可为空)

先加入 k 个元素,相当于保证每组至少一个,这样就转换为上一个问题,分组后再抽掉

$$\binom{n+k-1}{k-1} = \binom{n+k-1}{n}$$

4.2.1.3. 组合数性质/二项式推论

1.

$$\binom{n}{m} = \binom{n}{n-m}$$

所选集合对全集取补集(对称性)

n 取 m = n 剩 n-m = n 取 n-m

2.

$$\binom{n}{k} = \frac{n}{k}\binom{n-1}{k-1}$$

由定义得

3.

$$\binom{n}{m} = \binom{n-1}{m} + \binom{n-1}{m-1}$$

组合数递推式(杨辉三角公式)

4.2.2. 容斥原理
容斥系数:-1^k

4.2.3. 斯特林数
第二类斯特林数

n 个不同的元素,分成 m 个集合的方案数

- 递推式

$s(n,m) = s(n-1,m) + (m-1) * s(n-1,m)$

多出来的一个元素，要么放进一个新的集合，要么放进原有的一个集合当中

- 通项公式

先考虑一个弱化的情况，n 个不同元素，放入 m 个不同集合，集合可为空

m^n

从集合不同到集合相同，只需要再除以 m!即可

利用容斥，再变成-1 个空盒+2 空盒-3 个空盒。。。

$s(n,m) = \frac{1}{m!}\sum_{k=0}^{m} -1^k \binom{m}{k}(m-k)^n$

4.2.4. 范德蒙卷积
$\sum_{i=0}^{k}\binom{n}{i}\binom{m}{k-i} = \binom{n+m}{k}$
证明:

1.组合意义

从大小为$n+m$的集合中取出k个数,可以等效于把集合拆成两个集合,大小分别为n与m,然后从n中取i个数,从m中取出$k-i$个数的方案数.

因为枚举了i,所以只需要考虑一种拆法,不同拆法之间是等效的.

2.二项式定理

不懂,待补

4.2.5. 卡特兰数
长度为 n 的合法括号序列的数量,特别的,$H_1 = 1$

定义式

$$H_n = \sum_{i=0}^{n-1} H_i H_{n-i-1}, H_0 = 1$$

相关式子

$$H_n = \binom{2n}{n} - \binom{2n}{n-1}$$

$$H_n = \frac{\binom{2n}{n}}{n+1}$$

$$H_n = \frac{4n-2}{n+1} H_{n-1}$$

4.3. 博弈论

通过性质将大问题化简为小问题(n->n-2)

Who is HB?

4.4. 线性代数

4.4.1. 矩阵

- 线性运算+/-/数乘
- 乘法

封装模板

```
struct Mat
{
    int sz;
    vector<vector<ll>> a;
    Mat(int _sz) : sz(_sz), a(_sz + 10, vector<ll>(_sz + 10, 0)) {}
    Mat operator-(const Mat &T) const
    {
        Mat res(sz);
        for (int i = 1; i <= sz; i++)
            for (int j = 1; j <= sz; j++)
                res.a[i][j] = (a[i][j] - T.a[i][j]) % mod;
        return res;
    }
    Mat operator+(const Mat &T) const
    {
        Mat res(sz);
        for (int i = 1; i <= sz; i++)
            for (int j = 1; j <= sz; j++)
                res.a[i][j] = (a[i][j] + T.a[i][j]) % mod;
```

```
        return res;
    }
    Mat operator*(const Mat &T) const
    {
        Mat res(sz);
        for (int i = 1; i <= sz; i++)
            for (int k = 1; k <= sz; k++)
                if (a[i][k])
                    for (int j = 1; j <= sz; j++)
                        if (T.a[k][j])
                            res.a[i][j] += (a[i][k] * T.a[k][j]) %
mod, res.a[i][j] %= mod;
        return res;
    }
    Mat operator^(ll x) const
    {
        Mat res(sz), bas(sz);
        for (int i = 1; i <= sz; ++i)
            res.a[i][i] = 1;
        for (int i = 1; i <= sz; ++i)
            for (int j = 1; j <= sz; ++j)
                bas.a[i][j] = a[i][j] % mod;
        while (x)
        {
            if (x & 1)
                res = res * bas;
            bas = bas * bas;
            x >>= 1;
        }
        return res;
    }
};
```

4.5. 高等数学

4.5.1. 调和级数

$$\sum_{i=1}^{x} \frac{1}{i} = O(\ln x)$$

$$\sum_{i=1}^{n} \frac{n}{i} = O(n \ln x)$$

4.6. 结论

1. 有 s 种不同类型的糖果,总数为 n,两个不同的糖果可以配对成一组,问最多配对多少组.

如果数量最多的糖果不超过总数的一半,那我们总是可以配对出$\lfloor \frac{n}{2} \rfloor$组.

否则我们就无法用完最大的糖果,假设最大的糖果有 mx 个,那我们只能配对出 n-mx 组.

求方案:按照数量排序，先放奇数位，再放偶数位

2.一个数组和为 s，最多有$\sqrt{s}$种不同的数

3.$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$

5. 图论

5.1. 基础概念

5.1.1. 基环树

一张无向连通图包含恰好一个环(普通基环树)

一张有向弱连通图每个点的入度都为 1(基环外向树)

一张有向弱连通图每个点的出度都为 1(基环内向树)

5.2. 图的存储

链式前行星

```
void add(int a, int b) // a->b
{
    e[idx] = b, ne[idx] = h[a], h[a] = idx++;
}
for (int i = h[u]; i != -1; i = ne[i])//遍历
{
    int to = e[i];
    dfs(to);
}
```

5.3. 图论 DFS

- dfs 序

树 dfs 时经过节点的顺序,就是 dfs 序(dfn,是节点在 dfs 序中的编号)

性质:

祖先一定出现在后代之前

- 欧拉序

在 dfs 序的基础上,每一个子节点访问后都要记录自己

性质:

1.节点 x 第一次出现与最后一次出现之间的所有节点都在 x 的子树中

2.任意两个节点的 LCA 是欧拉序中两节点第一次出现位置中深度最小的节点

5.4. 树上问题

5.4.1. 树的直径

树上两点的最长距离

1.dfs

定理：在一棵树上，从任意节点 y 开始进行一次 DFS,到达的距离其最远的节点 z 必为直径的一端。

若存在负权边，则无法使用两次 DFS 的方式求解直径。

```
int n, c;
vector<int> edge[N];
int d[N];
void dfs(int u, int fa)
{
    for (auto to : edge[u])
        if (to != fa)
        {
            d[to] = d[u] + 1;
            if (d[to] > d[c])
                c = to;
            dfs(to, u);
        }
}
void solve()
{
    cin >> n;
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        edge[u].push_back(v), edge[v].push_back(u);
    }
    dfs(1, 0);
    d[c] = 0, dfs(c, 0);
    cout << d[c];
}
```

2.树形 dp

```

```

```
int n, d;
vector<int> edge[N];
int d1[N], d2[N];
void dfs(int u, int fa)
{
    d1[u] = d2[u] = 0;
    for (auto to : edge[u])
        if (to != fa)
        {
            dfs(to, u);
            int t = d1[to] + 1;
            if (t > d1[u])
                d2[u] = d1[u], d1[u] = t;
            else if (t > d2[u])
                d2[u] = t;
        }
    d = max(d, d1[u] + d2[u]);
}
void solve()
{
    cin >> n;
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        edge[u].push_back(v), edge[v].push_back(u);
    }
    dfs(1, 0);
    cout << d;
}
```

5.4.2. 最近公共祖先(Lowest Common Ancestor)

1. 倍增求 lca

时间复杂度:

预处理:$O(n \log n)$

询问:$O(\log n)$

```
int n, m;
vector<int> edge[N];
int f[N][21], dist[N];
//如果题目没有告诉你谁是父谁是子
// void dfs(int u, int fa)
// {
```

```
//      f[u][0] = fa;
//      for (auto to : edge[u])
//          if (to != fa)
//          {
//              dist[to] = dist[u] + 1;
//              dfs(to, u);
//          }
// }
void dfs(int x)
{
    for (auto i : edge[x])
    {
        dist[i] = dist[x] + 1;
        dfs(i);
    }
}
//预处理前要先确保有 f[y][0] = x;
dfs(1);
for (int i = 1; i <= 20; i++)//如果数据很大，这里要开大一些
    for (int j = 1; j <= n; j++)
    {
        if (f[j][i - 1])
            f[j][i] = f[f[j][i - 1]][i - 1];
    }
```

```
int lca(int u,int  v)
{
    if (dist[u] < dist[v])
            swap(u, v);
        int cha = dist[u] - dist[v];
        for (int j = 0; j <= 20 && cha; j++, cha /= 2)
            if (cha & 1)
                u = f[u][j];
        if (u != v)
        {
            for (int j = 20; j >= 0; j--)
                if (f[u][j] != f[v][j])
                    u = f[u][j], v = f[v][j];
            return f[u][0];
        }
    return u;
}
```

2. dfn 求 lca

时间复杂度:

预处理:$O(n\log n)$

询问:$O(1)$

查询[dfn[x]+1,dfn[y]]内深度最小的节点的父亲,就是 x,y 的 lca

比较时，取时间戳较小的结点也是正确的，不用记录深度

因为询问区间的 lca 的 dfn 一定小于区间内所有节点的 dfn

```
int f[20][N];
void solve()
{
    int n, q, s, cnt = 0;
    cin >> n >> q >> s;
    vector<vector<int>> e(n + 1);
    vector<int> dfn(n + 1, 0);
    auto get = [&](int x, int y) -> int
    { return dfn[x] < dfn[y] ? x : y; };
    auto dfs = [&](auto &dfs, int u, int fa) -> void
    {
        f[0][dfn[u] = ++cnt] = fa;
        for (auto to : e[u])
            if (to != fa)
                dfs(dfs, to, u);
    };
    auto lca = [&](int u, int v) -> int
    {
        if (u == v)
            return u;
        if ((u = dfn[u]) > (v = dfn[v]))
            swap(u, v);
        int d = __lg(v - u++);
        return get(f[d][u], f[d][v - (1 << d) + 1]);
    };
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        e[u].push_back(v), e[v].push_back(u);
    }
    dfs(dfs, s, 0);
    for (int i = 1; i <= __lg(n); i++)
        for (int j = 1; j + (1 << i) - 1 <= n; j++)
            f[i][j] = get(f[i - 1][j], f[i - 1][j + (1 << i - 1)]);
    for (int i = 1; i <= q; i++)
    {
        int u, v;
```

```
        cin >> u >> v;
        cout << lca(u, v) << endl;
    }
}
```

3. tarjan 求 lca

预处理:$O(n)$

询问:$O(mA(m+n,n)+n)$

A 是阿克曼函数的反函数,很小,视为常数,10^80 才为 4,2047 时为 3

离线处理所有询问

把点分成三类:

v=0:未访问过的点

v=1:访问过一次的点

v=2:访问过,并且已经回溯的点

把所有询问离线下来,挂到对应的节点上形成一个二元组{to,i}

只有当 v[to]=2 时,此时 lca 就是 to 所在并查集指向的节点

注意特判 u=v 的情况,tarjan 无法处理这种情况

```
auto tarjan = [&](auto &tarjan, int u) -> void
{
    v[u] = 1;
    for (auto to : e[u])
    {
        if (v[to])
            continue;
        tarjan(tarjan, to);
        p[to] = u;
    }
    for (auto [to, i] : q[u])
    {
        if (v[to] == 2)
            ans[i] = find(find, to);
    }
    v[u] = 2;
};
```

4.树剖求 lca

5.欧拉序求 lca

完全被 dfn lca 吊打,不用学

5.4.3. 虚树

二次排序+lca

时间复杂度:$O(m\log n)$m 为关键点数,n 为总点数

```
auto build = [&]() -> void
{
    sort(a.begin() + 1, a.begin() + num + 1, [&](int a, int b)
            { return dfn[a] < dfn[b]; });//将关键点按照 dfn 排序
    c.push_back(1);
    for (int i = 1; i < num; i++)
    {
        c.push_back(a[i]);
        c.push_back(lca(a[i], a[i + 1]));
    }
    c.push_back(a[num]);
    sort(c.begin() + 1, c.end(), [&](int a, int b)
            { return dfn[a] < dfn[b]; });//虚树上的所有点按 dfn 排序
    c.erase(unique(c.begin(), c.end()), c.end());//去重
    for (int i = 0; i + 1 < c.size(); i++)
    {
        ll lc = lca(c[i], c[i + 1]);
        ll d = cal(lc, c[i + 1]);
        ne[lc].push_back({c[i + 1], d});//;连边构建虚树
        ne[c[i + 1]].push_back({lc, d});
    }
};
```

5.5. 最短路

5.5.1. dijkstra

只适用与非负权图!

- 朴素版

时间复杂度:$O(\log n^2 + m)$

```
struct node
{
    int y, v;
    node(int _y, int _v)
    {
        y = _y;
        v = _v;
    }
};
bool used[N];
vector<node> edge[M];
int n, m, k, dist[N];
int dijkstra(int s, int t)
{
    memset(used, false, sizeof used);
    memset(dist, 127, sizeof dist);
    dist[s] = 0;
    while (true)
    {
        int x = -1;
        for (int i = 1; i <= n; i++)
            if (!used[i] && dist[i] < 1 << 30)
                if (x == -1 || dist[i] < dist[x])
                    x = i;
        if (x == t || x == -1)
            break;
        used[x] = true;
        for (auto i : edge[x])
            dist[i.y] = min(dist[i.y], dist[x] + i.v);
    }
    if (dist[t] < 1 << 30)
        return dist[t];
    return -1;
}
```

- 堆优化

时间复杂度:$O((n+m)\log m)$

```
struct node
{
    int y, v;
    node(int _y, int _v)
    {
        y = _y;
        v = _v;
```

```
    }
};
vector<node> edge[N];
bool st[N];
int dist[N];
int n, m, s;
void dijkstra(int s)
{
    memset(dist, 0x3f, sizeof dist);
    memset(st, false, sizeof st);
    priority_queue<pii, vector<pii>, greater<pii>> q;
    dist[s] = 0;
    q.push({0, s});
    while (q.size())
    {
        auto t = q.top();
        q.pop();
        if (st[t.y])
            continue;
        st[t.y] = true;
        for (auto i : edge[t.y])
            if (dist[t.y] + i.v < dist[i.y])
                dist[i.y] = dist[t.y] + i.v, q.push({dist[i.y],
i.y});
    }
}
```

5.5.2. floyd

时间复杂度:$O(n^3)$

```
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
        {
            dist[i][j] = 1e18;
            if (i == j)
                dist[i][j] = 0;
        }
    for (int k = 1; k <= n; k++)
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
                if (dist[i][k] < 1e18 && dist[k][j] < 1e18)
                    dist[i][j] = min(dist[i][j], dist[i][k] +
dist[k][j]);
```

5.6. 最小生成树(Minimum Spanning Tree,MST)

- krukskal 算法

时间复杂度:$O(m\log m)$

```
struct Node
{
    int x, y, v;
    bool operator<(const Node &A) const
    {
        return v < A.v;
    }
} a[M];
int n, m, fa[N];
void init()
{
    for (int i = 1; i <= n; i++)
        fa[i] = i;
}
int find(int x)
{
    if (x != fa[x])
        return fa[x] = find(fa[x]);
    return fa[x];
}
int kruskal()
{
    init();
    sort(a + 1, a + 1 + m);
    int ans = 0, cnt = n;
    for (int i = 1; i <= m; i++)
    {
        int x = find(a[i].x), y = find(a[i].y);
        if (x != y)
            fa[x] = y, ans += a[i].v, cnt--;
        if (cnt == 1)
            break;
    }
    if (cnt != 1)
        return -1;
    return ans;
}
```

- prim 算法

```
int n, m, dist[N];
bool st[N];
struct Node
```

```
{
    int y, w;
    Node(int _y, int _w) { y = _y, w = _w; };
};
vector<Node> edge[N];
int prim()
{
    memset(st, false, sizeof st);
    memset(dist, 127, sizeof dist);
    dist[1] = 0;
    int ans = 0, num = 0;
    while (true)
    {
        int x = -1;
        for (int i = 1; i <= n; i++)
            if (!st[i] && dist[i] < (1 << 30))
                if (x == -1 || dist[i] < dist[x])
                    x = i;
        if (x == -1)
            break;
        num++;
        ans += dist[x];
        st[x] = true;
        for (auto c : edge[x])//每条边只会被枚举两次。
            dist[c.y] = min(dist[c.y], c.w);
    }
    if (num != n)
        return -1;
    else
        return ans;
}
```

5.7. 最短路与最小生成树的区别

5.7.1. 最短路

bfs:边权为 0/1

拓扑+dp: DAG 每次找无法再被更新的点来更新其他点

dijkstra: 边权非负 贪心,每次取离起点最近的

spfa:暴力 更新到无法被更新为止

dijkstra vs prim

距离起点最近 距离点集最近

5.8. 拓扑排序(Topological sorting)

时间复杂度:$O(n+m)$

```
bool toposort()
{
    for (auto i : all)
        if (!in[i])
        {
            q.push(i);
            st[i] = false;
        }
    while (q.size())
    {
        auto t = q.front();
        q.pop();
        L.push(t);//cun'fang
        for (auto i : edge[t])
        {
            if (--in[i] == 0)
                q.push(i);
        }
    }
    return L.size() ==n;
}
```

5.9. 二分图

5.9.1. 二分图最大匹配

```
int n;
vector<vector<int>> e(n + 1);
vector<int> got(n + 1, 0), vis(n + 1, 0);
for (int i = 1; i <= n; i++)
{
    auto dfs = [&](auto dfs, int u) -> bool
    {
        for (auto to : e[u])
            if (!vis[to])
            {
                vis[to] = 1;
                if (!got[to] || dfs(dfs, got[to]))
                {
                    got[to] = u;
                    return true;
                }
            }
```

```
        return false;
    };
    for (int j = 0; j <= n; i++)
        vis[j] = 0;
    dfs(dfs, i);
}
```

5.10. 点双/边双

5.10.1. 无向图

割点: 对于一个点 x,删去该点及所有相连的边,图不再联通

割边/桥: 对于一条边 e,删去该边,图不再联通

点双连通图:一个图不存在割点

边双连通图:一个图不存在割边

点双联通分量: 极大点双联通子图

边双联通分量: 极大边双联通子图

5.10.2. tarjan

若某个点是割点,一定满足以下一种情况:

1.子树中不存在跨越该点,连向该点上面的点的边

dfn[u]≤low[v]

2.该点是根且有多个儿子

若某个点是割边,一定满足:

子树中不存在跨越该点,连向该点上面的点的边

dfn[u]<low[v]

tarjan 求割点

```
int dfn[N], low[N], boo[N], cnt = 0, root;
vector<int> e[N];
void tarjan(int u)
{
    dfn[u] = low[u] = ++cnt;
    int num = 0;
    for (auto to : e[u])
        if (!dfn[to])//树边
        {
            tarjan(to);
```

```
            low[u] = min(low[u], low[to]);
            if (low[to] >= dfn[u])
            {
                num++;
                if (u != root || num > 1)
                    boo[u] = 1;
            }
        }
        else//返祖边
            low[u] = min(low[u], dfn[to]);
}
```

5.10.3. 有向图

5.11. 点双/边双

边双:删掉桥后,每个联通块都是一个边双联通分量

点双:一个割点可能属于多个点双联通分量

用栈维护

5.12. 强联通分量

有向图:

弱联通:单向可达

强联通:双向可达

强联通分量:极大强联通子图

求强联通分量:

1.kosaraju

对任意点作为起点进行一次 dfs

按照出栈顺序存放起来,在反图上 dfs

正向联通+反向联通=强联通

```

```

2.tarjan

```
vector<int> e[N];
int bel[N], vis[N], dfn[N], low[N], sz[N], idx, ans;
vector<vector<int>> scc;
vector<int> stk;
```

```
void tarjan(int u)
{
    dfn[u] = low[u] = ++idx;
    stk.push_back(u);
    vis[u] = 1;
    for (auto &to : e[u])
    {
        if (!dfn[to])
        {
            tarjan(to);
            low[u] = min(low[u], low[to]);
        }
        else if (vis[to])
            low[u] = min(low[u], dfn[to]);
    }
    if (dfn[u] == low[u])
    {
        ans++;
        while (true)
        {
            auto t = stk.back();
            stk.pop_back();
            vis[t] = 0;
            bel[t] = ans;
            sz[ans]++;
            if (t == u)
                break;
        }
    }
}
```

5.13. 网络流

5.13.1. 定义

源点,汇点,容量

可行流

增广路

残量网络

割/切

5.14. 同余最短路

https://www.luogu.com.cn/problem/P3951

求两个互质的数 a,b 最大的,不能凑出来的数

设$a < b$

那么在 %a 意义下,所有能凑出来的数可以表示为$k * a + i(1 \le i \le a - 1)$

按照 i 分类,分为 0,1,...,a-1

在 %a 意义下,a 的系数就不重要了,接下来考虑 b

设 f[i]表示,%a 为 i,最小的,能凑出来的数

f[i]一定是从某个 x 转移过来的,f[i]=f[(x+b)%a]

显然 f[0]=0,那么从 0 开始跑最短路,就可以得到每个位置最小能凑出来的数

因为 f[i]=k*a+i,k 是最小的,能表达出来的,%a=i 的数

那么$(k-1) * a + i$就是最大的,无法表示出来的,%a=i 的数

每个点只有一条出边,最后整个图是一个环

最大的 f[i]就是$(a-1) * b\%a$

所以答案就是 a*b-a-b

5.15. 优化建图技巧

5.15.1. 线段树优化建图

```
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
#define sum(x) (a[x].sum)
#define tag(x) (a[x].t)
#define ls(x) (a[x].l)
#define rs(x) (a[x].r)
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef double db;
const ll mod = 998244353;
const int N = 1e6 + 100, M = 5e5 + 10;
vector<pll> e[N];
int opt;
ll a[N], dist[N];
bool vis[N];
void build(int id, int l, int r)
{
    if (l == r)
    {
```

```
        a[l] = id;
        return;
    }
    int mid = l + r >> 1;
    e[id].push_back({id * 2, 0});
    e[id].push_back({id * 2 + 1, 0});
    // 反向边,用虚点建边
    e[id * 2 + M].push_back({id + M, 0});
    e[id * 2 + 1 + M].push_back({id + M, 0});
    build(id * 2, l, mid), build(id * 2 + 1, mid + 1, r);
}
void modify(int id, int l, int r, int ql, int qr, int v, int w)
{
    if (r < ql || l > qr)
        return;
    if (l >= ql && r <= qr)
    {
        if (opt == 2)
            e[v + M].push_back({id, w});
        else
            e[id + M].push_back({v, w});
        return;
    }
    int mid = l + r >> 1;
    modify(id * 2, l, mid, ql, qr, v, w);
    modify(id * 2 + 1, mid + 1, r, ql, qr, v, w);
}

void solve()
{
    int n, m, s;
    cin >> n >> m >> s;
    build(1, 1, n);
    for (int i = 1; i <= n; i++)
        e[a[i]].push_back({a[i] + M, 0}), e[a[i] +
M].push_back({a[i], 0});
    for (int i = 1; i <= m; i++)
    {
        cin >> opt;
        if (opt == 1)
        {
            ll u, v, w;
            cin >> u >> v >> w;
            e[a[u] + M].push_back({a[v], w});
        }
        else
```

```
        {
            ll x, l, r, w;
            cin >> x >> l >> r >> w;
            modify(1, 1, n, l, r, a[x], w);
        }
    }
    auto dij = [&](int s) -> void
    {
        for (int i = 1; i <= 1e6; i++)
            dist[i] = 1e18;
        dist[s] = 0;
        priority_queue<pll, vector<pll>, greater<pll>> pq;
        pq.push({0, s});
        while (pq.size())
        {
            auto [d, t] = pq.top();
            pq.pop();
            if (vis[t])
                continue;
            vis[t] = 1;
            for (auto [to, w] : e[t])
            {
                if (dist[to] > dist[t] + w)
                    dist[to] = dist[t] + w, pq.push({dist[to], to});
            }
        }
    };
    dij(a[s] + M);
    for (int i = 1; i <= n; i++)
        cout << (dist[a[i]] == 1e18 ? -1 : dist[a[i]]) << " ";
}
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    // cin >> _;
    while (_--)
        solve();
    return 0;
}
```

6. 动态规划

6.1. 背包 dp

6.1.1. 多重背包

- 二进制优化

时间复杂度:$nm\log v$ n 物品种类数 m 体积 v 该物品数量

```
int n, m;
int v[N], w[N], cnt[N], f[N];
void solve()
{
    cin >> n >> m;
    for (int i = 1; i <= n; i++)
        cin >> v[i] >> w[i] >> cnt[i];
    for (int i = 1; i <= n; i++)
    {
        int k = 1;
        for (k = 1; k <= cnt[i]; k *= 2)
        {
            cnt[i] -= k;
            for (int j = m; j >= k * v[i]; j--)
                f[j] = max(f[j], f[j - k * v[i]] + k * w[i]);
        }
        if (cnt[i])
            for (int j = m; j >= cnt[i] * v[i]; j--)
                f[j] = max(f[j], f[j - cnt[i] * v[i]] + cnt[i] *
w[i]);
    }
    cout << f[m] << endl;
}
```

6.2. dp 的优化

6.2.1. 斜率优化/凸壳优化

```

```

7. 计算几何

7.1. 基础知识

7.1.1. 点

```
struct Point
{
  int x, y;
  Point(int x, int y): x(x), y(y) {}
  bool operator<(Point P)const
```

```
  {
    if (x != P.x)
      retun x < P.x;
    return y < P.y;
  }
};
```

7.1.2. 向量

7.1.2.1. 点积(Dot product)

$\vec{a} *$

7.1.3. 直线方程

- 两点式

$$\frac{x - x_1}{x_2 - x_1} = \frac{y - y_1}{y_2 - y_1}$$

- 一般式

$$Ax + By + C = 0$$

点到直线距离公式

$$d = \frac{|Ax_0 + By_0 + C|}{\sqrt{A^2 + B^2}}$$

两平行线距离公式

$$d = \frac{|C_1 - C_2|}{\sqrt{A^2 + B^2}}$$

7.1.4. 点积

- 定义

两个点(ax,ay),(bx,by)，它们的点积 dot 定义为 $dot(a, b) = \vec{a} \cdot \vec{b} = ax * bx + ay * by = |a||b| \cos\theta$

θ为两向量的夹角 -应用 判断两个向量的夹角

dot (a,b)> 0, $0 \leq \theta < \frac{\pi}{2}$

dot (a,b)$= 0$, $\theta = \frac{\pi}{2}$

dot (a,b)< 0, $\frac{\pi}{2} < \theta \leq \pi$

7.1.5. 叉积

$cross(a, b) = \vec{a} \times \vec{b} = ax * by - ay * bx = |a||b| \sin\theta$

表示向量$\vec{a}$逆时针转到$\vec{b}$的角度

因此有$\vec{a} \times \vec{b} = -\vec{b} \times \vec{a}$

corss(a,b)>0 , b 在 a 的逆时针方向$(0 < \theta < \pi)$

corss(a,b)=0 , b,a 共线$(\theta = 0 \| \theta = \pi)$

corss(a,b)< 0 , b 在 a 的顺时针方向$(-\pi < \theta < 0)$

- 几何意义

表示两向量的有向面积

7.1.6. 极角

- 极坐标

用角度和长度描述位置的坐标系

- 极点

极坐标的原点

- 极轴

以极点为起点作射线的参考系

- 极经

点到极点的距离

- 极角

从极轴逆时针到极经所经过的角度 $\theta \in \{-\pi, \pi\}$

- 求极角

```
atan(y,x)//原点到(x,y)的向量与 x 轴的夹角(-pi,pi]
```

- 极角排序

利用极角序

```
bool cmp(Point a, Point b)
{
    if(dcmp(atan2(a.y, a.x) - atan2(b.y, b.x)) == 0) //dcmp 为判断浮点数是否为 0 的函数
        return a.x < b.x;
    return atan2(a.y, a.x) < atan2(b.y, b.x);
}
```

利用向量叉乘

```

```

```
struct P
{
    ll x, y;
    ll cross(P T) { return x * T.y - y * T.x; }
    ll quad()
    {
        if (x > 0 && y >= 0)
            return 1;
        if (x <= 0 && y > 0)
            return 2;
        if (x < 0 && y <= 0)
            return 3;
        if (x >= 0 && y < 0)
            return 4;
    }
};
struct node
{
    ll qd;
    P v;
    bool operator<(const node &T)
    {
        return (qd != T.qd ? qd < T.qd : v.cross(T.v) > 0);
    }
};
```

8. 字符串

8.1. 基础定义

子串,前缀(prefixs),后缀(suffixs)

Border:

存在一个长度 len,满足该长度的前缀和后缀相同,则称该前缀/后缀是该字符串的一个 border

周期: 存在一个p,使得$p < i \leq |S|$,满足$S[i] = S[i-p]$,则称 p 是 S 的一个周期

循环节:若周期$p \mid |S|$,则周期 p 是 S 的一个循环节

特殊性质:

1. p 是 S 的周期⇔|S|-p 是 S 的 Border

证明:

p 是 S 的周期⇔S[i-p]=S[i]

p 是 S 的 Border⇔S[1,q]=S[|S|-q+1,|S|]

S[1]=S[|S|-q+1]

S[2]=S[|S|-q+2]

2. Border 的传递性

S 的 Border 的 Border 的也是 S 的 Border border 树:

8.2. 哈希

8.2.1. 单哈希

```
struct Hash
{
    const ll base = 1361;
    const ll p = 998244353;
    int n;
    vector<ll> a, c;
    Hash(int _n, string arr) : n(_n), a(n + 10), c(n + 10, 0)
    {
        ll res = 0;
        c[0] = 1;
        for (int i = 1; i <= n; i++)
        {
            res = (res * base + arr[i]) % p;
            c[i] = c[i - 1] * base % p;
            a[i] = res;
        }
    }
    ll query(int l, int r) // 询问子串的哈希值
    {
        // if (l < 1 || r > n)
        //     cout << "invalid input" << endl;
        ll ans = (a[r] - a[l - 1] * c[r - l + 1] % p + p) % p;
        return ans;
    }
};
```

8.2.2. 双哈希

```
struct Hash
{
    const ll base1 = 1361, base2 = 1301;
    const ll p1 = 1000000123, p2 = 1000000021;
    int n;
    vector<ll> a1, a2, c1, c2;
```

```
    Hash(int _n) : n(_n), a1(n + 10), a2(n + 10), c1(n + 10, 0),
c2(n + 10) {}
    void build(int n, vector<char> arr)
    {
        ll res = 0;
        c1[0] = 1, c2[0] = 1;
        for (int i = 1; i <= n; i++)
        {
            res = (res * base1 + arr[i]) % p1;
            c1[i] = c1[i - 1] * base1 % p1;
            a1[i] = res;
        }
        res = 0;
        for (int i = 1; i <= n; i++)
        {
            res = (res * base2 + arr[i]) % p2;
            c2[i] = c2[i - 1] * base2 % p2;
            a2[i] = res;
        }
    }
    pll query(int l, int r) // 询问子串的哈希值
    {
        ll hashval1 = (a1[r] - 1ll * a1[l - 1] * c1[r - l + 1] % p1
+ p1) % p1;
        ll hashval2 = (a2[r] - 1ll * a2[l - 1] * c2[r - l + 1] % p2
+ p2) % p2;
        return {hashval1, hashval2};
    }
};
```

8.2.3. 带修哈希

用树状数组维护即可

```

```

8.2.4. 树哈希

8.3. KMP

next 数组:

next[i]=Preffix[i]的非平凡的最大 Border(非平凡:除掉自己)

8.3.1. kmp 求 border

```

```

```
vector<int> nxt(N, 0);
void kmp(vector<char> &s, int n)
{
    nxt[1] = 0;
    for (int i = 2; i <= n; i++)
    {
        nxt[i] = nxt[i - 1];
        while (nxt[i] && s[i] != s[nxt[i] + 1])
            nxt[i] = nxt[nxt[i]];
        nxt[i] += (s[i] == s[nxt[i] + 1]);
    }
}
```

8.3.2. kmp 匹配

8.4. Manacher

```
struct Manacher
{
    // 设插入 $ 后的字符串为 s',那么 s'与原串 s 存在以下关系:
    // s'的极长回文子串一定为奇数(因为一定以 $ 结尾)
    //|s'|=2|s|+1
    //|s|=(|s'|-1)/2
    //|s|=|s'|/2(下取整)
    int m;
    vector<char> a;
    vector<int> p; // p[i]表示以 i 为中心的最长回文子串
    Manacher(int _n) : a(2 * _n + 10), p(2 * _n + 10) {}
    void build(int n, vector<char> &s)
    {
        m = 0;
        a[++m] = '$';
        for (int i = 1; i <= n; i++)
            a[++m] = s[i], a[++m] = '$';
        int M = 0, R = 0;
        for (int i = 1; i <= m; i++)
        {
            if (i > R)
                p[i] = 1;
            else
                p[i] = min(p[2 * M - i], R - i + 1);
            while (i - p[i] > 0 && i + p[i] - 1 <= m && a[i - p[i]]
== a[i + p[i]]) // 暴力向右拓展
                ++p[i];
            if (i + p[i] - 1 > R) // 更新右端点最右的回文串的回文中心,回文
```

```
半径
                M = i, R = i + p[i] - 1;
        }
    }
    int query()
    {
        int ans = 0;
        for (int i = 1; i <= m; i++)
            ans = max(ans, p[i] - 1);
        return ans;
    }
};
```

9. 杂项

9.1. 时间复杂度分析

- 势能分析法

9.2. 随机

```
mt19937_64 rnd(chrono::duration_cast<chrono::nanoseconds>
(chrono::system_clock::now().time_since_epoch()).count());
void solve()
{
    ll l, r;
    cin >> l >> r;
    cout << rnd() << endl;
    cout << rnd() % (r - l + 1) + l << endl;
}
```

9.3. 龟速乘

适用于两个 ll 级别的数乘法取模

```
ll qmul(ll a, ll b)
{
    ll ans = 0;
    while (b)
    {
        if (b & 1)
            ans = (ans + a) % mod;
        a = a * 2 % mod;
        b >>= 1;
    }
```

```
    return ans;
}
```

9.4. 常数优化

9.4.1. 快速取模

当模数是从输入中给定的

```
struct fastmod
{
    using u64 = uint64_t;
    using u128 = __uint128_t;
    int f, l;
    u64 m, d;
    fastmod(u64 d) : d(d)
    {
        l = 64 - __builtin_clzll(d - 1);
        const u128 one = 1;
        u128 M = ((one << (64 + l)) + (one << l)) / d;
        if (M < (one << 64))
            f = 1, m = M;
        else
            f = 0, m = M - (one << 64);
    }
    friend u64 operator/(u64 n, const fastmod &m)
    { // get n / d
        if (m.f)
            return u128(n) * m.m >> 64 >> m.l;
        else
        {
            u64 t = u128(n) * m.m >> 64;
            return (((n - t) >> 1) + t) >> (m.l - 1);
        }
    }
    friend u64 operator%(u64 n, const fastmod &m)
    { // get n % d
        return n - n / m * m.d;
    }
};
ull p;
cin >> p;
fastmod a(p);
ull n = 1e9 + 7;
cout << n % a;
```

9.4.2. 快读

```
int read()
{
    int x = 0, f = 1;
    char ch = getchar();
    while (ch < '0' || ch > '9')
    {
        if (ch == '-')
            f = -1;
        ch = getchar();
    }
    while (ch >= '0' && ch <= '9')
    {
        x = x * 10 + ch - 48;
        ch = getchar();
    }
    return x * f;
}
```

9.5. 离线算法

9.5.1. 莫队

9.5.1.1. 适用范围

- 询问区间信息
- 可以离线算法
- 难以高速合并区间
- 可以高速拓展到 l/r +−1

9.5.1.2. 时间复杂度

把整个区间分成$\frac{n}{B}$块,q 为询问数,n 为区间长度

将所有询问区间离线,排序(第一关键字,l/B,第一关键字,r)

左端点: B*q(每个块的左端点一次最多移动块长 B)

右端点:$\frac{n}{B} * n$(每个块的右端点加起来移动 n)

总时间复杂度:$O\left(B * q + \frac{n}{B} * n\right)$

令 B=$\frac{n}{\sqrt{q}}$,变成$O(n\sqrt{q})$

9.5.1.3. 奇偶排序优化

使右端点在不同块区间呈峰形移动

```
[&](array<ll, 3> a, array<ll, 3> b)
{
    int c=a[0]/B;
    if(c!=b[0]/B)
        return c<b[0]/B;
    return c%2?a[1]>b[1]:a[1]<b[1];
};
```

P1494 [国家集训队] 小 Z 的袜子

```
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
#define stop cout << "?" << '\n'
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
typedef pair<long long, long long> pll;
const int N = 5e4 + 10, M = 1e4 + 10;
const ll mod = 1e9 + 7;
const double pi = acos(-1);
int n, q;
ll c[N], cnt[N], ans[N], ans2[N];
array<ll, 3> que[N];
void solve()
{
    cin >> n >> q;
    for (int i = 1; i <= n; i++)
        cin >> c[i];
    for (int i = 1; i <= q; i++)
    {
        ll l, r;
        cin >> l >> r;
        que[i] = {l, r, i};
        ans2[i] = (r - l + 1) * (r - l) / 2;
    }
    int B = 500;
    sort(que + 1, que + q + 1, [&](array<ll, 3> a, array<ll, 3> b)
         {
        int c=a[0]/B;
        if(c!=b[0]/B)
        return c<b[0]/B;
        return c%2?a[1]>b[1]:a[1]<b[1]; });
    ll l = 1, r = 0, tmp = 0;
    auto add = [&](int x)
```

```
    {
        tmp += cnt[c[x]];
        cnt[c[x]]++;
    };
    auto del = [&](int x)
    {
        cnt[c[x]]--;
        tmp -= cnt[c[x]];
    };
    for (int i = 1; i <= q; i++)
    {
        while (r < que[i][1])
            r++, add(r);
        while (l > que[i][0])
            l--, add(l);
        while (r > que[i][1])
            del(r), r--;
        while (l < que[i][0])
            del(l), l++;
        ans[que[i][2]] = tmp;
    }
    for (int i = 1; i <= q; i++)
    {
        ll d = gcd(ans[i], ans2[i]);
        if (ans[i])
            cout << ans[i] / d << "/" << ans2[i] / d << endl;
        else
            cout << "0/1" << endl;
    }
}
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
    int _ = 1;
    // cin >> _;
    while (_--)
        solve();
    return 0;
}
```

P2709 小 B 的询问

9.5.2. 整体二分

[模板]可持久化线段树 2

给定一个数组 a,有 q 个询问，每次询问区间[l,r]内的第 k 小

先考虑只有一个询问,二分答案,check 是否有 k-1 个小于等于的数

如果有多个询问,则需要引入整体二分

```
struct BIT
{
    int n;
    vector<ll> a;
    BIT(int _n) : n(_n + 3), a(n + 1) {}
    int lb(int x) { return x & -x; }
    void add(int x, int y)
    {
        for (; x < n; x += lb(x))
            a[x] += y;
    }
    ll query(int x)
    {
        ll res = 0;
        for (; x; x ^= lb(x))
            res += a[x];
        return res;
    }
};
struct node
{
    ll x, y, k, id;
    bool typ;
};
void solve()
{
    int n, m, tp = 0;
    cin >> n >> m;
    BIT bit(n);
    vector<node> q(n + m + 1), q1(n + m + 1), q2(n + m + 1);
    vector<ll> a(n + 1), ans(m + 1);
    for (int i = 1; i <= n; i++)
    {
        cin >> a[i];
        q[++tp] = {a[i], 1, inf, i, 0};
    }
    for (int i = 1; i <= m; i++)
    {
        ll l, r, k;
        cin >> l >> r >> k;
        q[++tp] = {l, r, k, i, 1};
```

```
    }
    auto dfs = [&](const auto &dfs, ll l, ll r, ll L, ll R) -> void
    {
        if (l > r)
            return;
        if (L == R)
        {
            for (ll i = l; i <= r; i++)
                if (q[i].typ)
                    ans[q[i].id] = L;
            return;
        }
        ll mid = (L + R) >> 1, cnt1 = 0, cnt2 = 0;
        for (ll i = l; i <= r; i++)
        {
            if (!q[i].typ)
            {
                if (q[i].x <= mid)
                {
                    bit.add(q[i].id, 1);
                    q1[++cnt1] = q[i];
                }
                else
                    q2[++cnt2] = q[i];
            }
            else
            {
                ll res = bit.query(q[i].y) - bit.query(q[i].x - 1);
                if (res >= q[i].k)
                    q1[++cnt1] = q[i];
                else
                {
                    q[i].k -= res;
                    q2[++cnt2] = q[i];
                }
            }
        }
        for (ll i = 1; i <= cnt1; i++)
            if (!q1[i].typ)
                bit.add(q1[i].id, -1);
        for (ll i = 1; i <= cnt1; i++)
            q[i + l - 1] = q1[i];
        for (ll i = 1; i <= cnt2; i++)
            q[i + l + cnt1 - 1] = q2[i];

        dfs(dfs, l, l + cnt1 - 1, L, mid);
```

```
        dfs(dfs, l + cnt1, r, mid + 1, R);
    };
    dfs(dfs, 1, tp, -inf, inf);
    for (int i = 1; i <= m; i++)
        cout << ans[i] << endl;
}
```

9.5.3. 离线区间询问

- 区间颜色数

P1972 [SDOI2009] HH 的项链

```
struct BIT
{
    int n;
    vector<ll> a;
    BIT(int _n) : n(_n + 3), a(n + 1) {}
    int lb(int x) { return x & -x; }
    void add(int x, int y)
    {
        for (; x < n; x += lb(x))
            a[x] += y;
    }
    ll query(int x)
    {
        ll res = 0;
        for (; x; x ^= lb(x))
            res += a[x];
        return res;
    }
};
struct query
{
    int l, r, id;
    bool operator<(const query &T) const { return r < T.r; };
};
void solve()
{
    int n, m;
    cin >> n;
    BIT bit(n);
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    cin >> m;
    vector<query> q(m);
```

```
    vector<int> ans(m), mp(N, 0);
    for (int i = 0; i < m; i++)
    {
        cin >> q[i].l >> q[i].r;
        q[i].id = i;
    }
    sort(q.begin(), q.end());
    int pos = 1;
    for (auto [l, r, id] : q)
    {
        for (int i = pos; i <= r; i++)
        {
            if (mp[a[i]])
                bit.add(mp[a[i]], -1);
            bit.add(i, 1);
            mp[a[i]] = i;
        }
        pos = r + 1;
        ans[id] = bit.query(r) - bit.query(l - 1);
    }
    for (auto i : ans)
        cout << i << endl;
}
```

- 区间 mex

P4137 Rmq Problem / mex

```

```

9.6. 置换环

应用:

求将一个排列变有序需要的最少操作次数

对每个节点，将其指向其排序后应该放到的位置，这样首尾相接形成了多个环。

答案为$\sum_{i=1}^{k} sz[i]-1$,即所有环的长度-1 求和

```
for (int i = 1; i <= n; i++)
{
    if(st[i])
        continue;
    int res = 0;//置换环的大小
    int x = i;
    while (!st[x])
    {
```

```
        st[x] = true;
        x = to[x];
        res++;
    }
    ans += res - 1;
}
```

9.7. 开根号

```
ll safe_sqrt(long long a)
{
    ll ret = sqrt(a);
    while ((ret + 1) * (ret + 1) <= a)
        ret++;
    while (ret * ret > a)
        ret--;
    return ret;
}
```

9.8. 矩阵旋转

顺时针旋转90°

```
void Rotate()
{
    for (int i = 1; i <= (n - 1) / 2 + 1; i++)
        for (int j = 1; j <= n / 2; j++)
        {
            int num = a[i][j];
            a[i][j] = a[n - j + 1][i];
            a[n - j + 1][i] = a[n - i + 1][n - j + 1];
            a[n - i + 1][n - j + 1] = a[j][n - i + 1];
            a[j][n - i + 1] = num;
        }
}
```